

AI224: Operating Systems — Lecture 11

Dr. Karim Lounis

Jan - May 2025



Memory Management

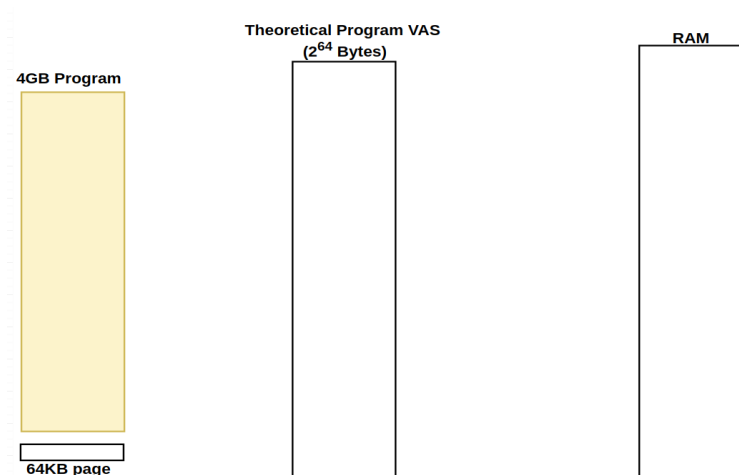
Memory Management (Con't)

Three-level paging (Graphical example)

Consider a 64-bit computer that uses 64KB for page sizes and 4 Bytes for PTE. A program that needs to be executed requires 4GB of memory. What will be the page tables tree for that program –process?

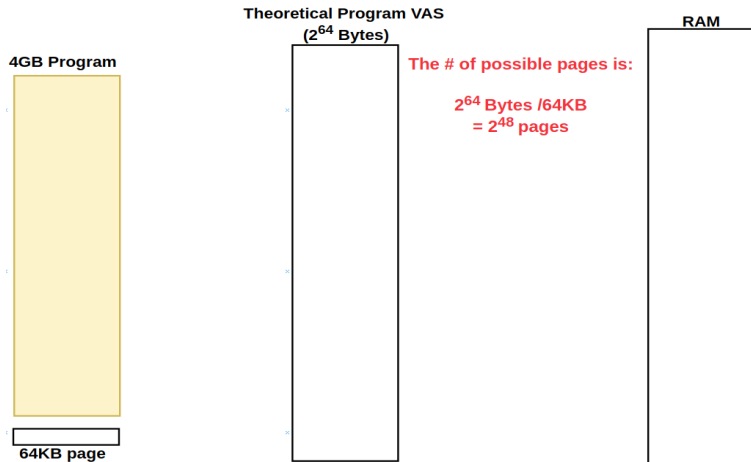
Three-level paging (Graphical example)

1. Compute the # of possible pages.



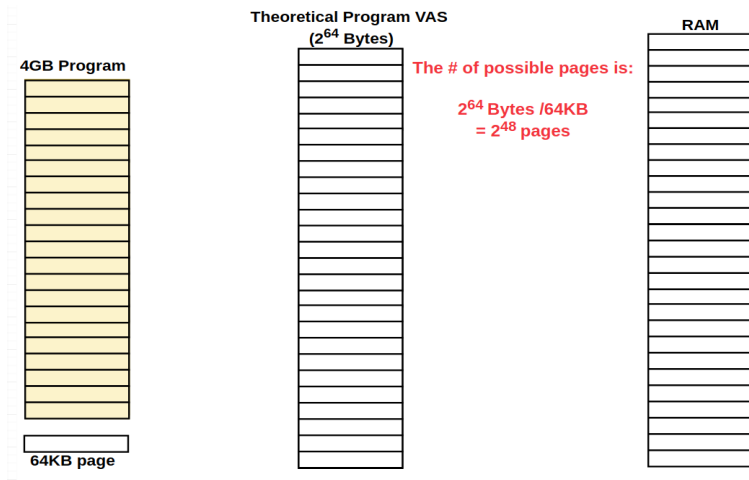
Three-level paging (Graphical example)

1. Compute the # of possible pages.



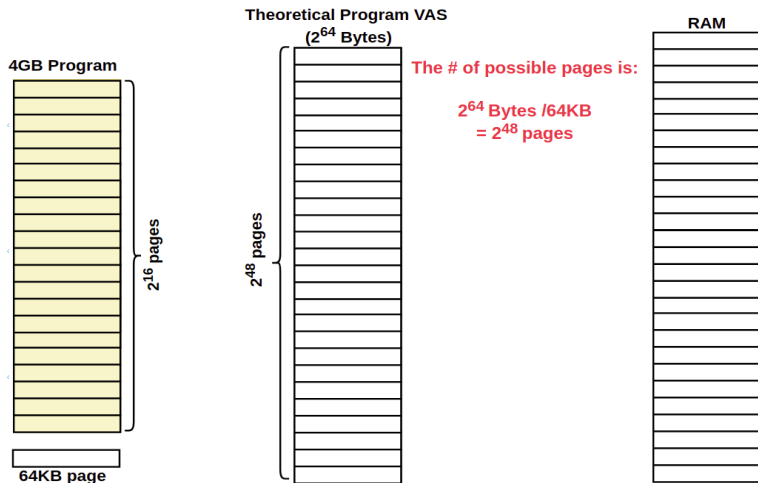
Three-level paging (Graphical example)

2. Divide virtual and physical memory space into pages and frames respectively.



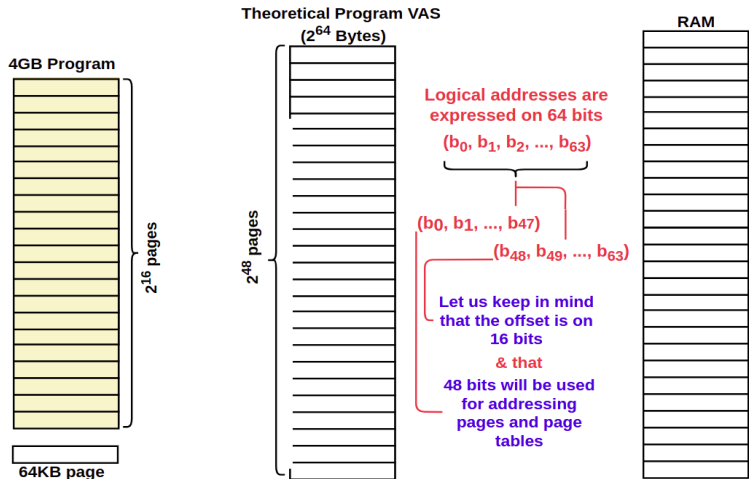
Three-level paging (Graphical example)

2. Divide virtual and physical memory space into pages and frames respectively.



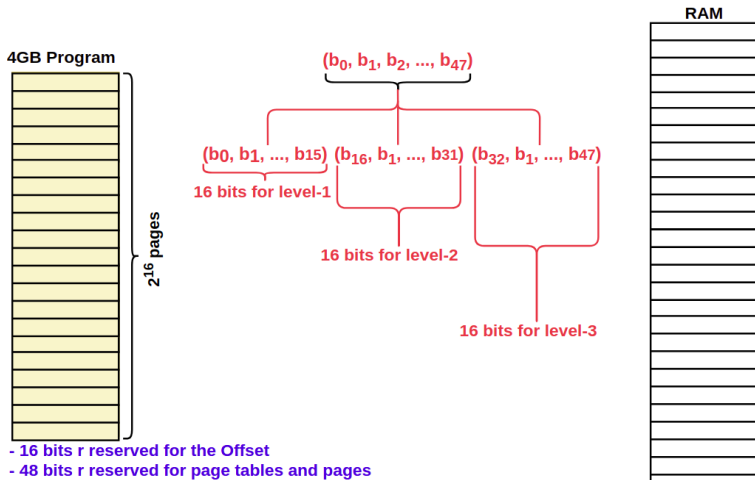
Three-level paging (Graphical example)

3. Analyze the logical address parts.



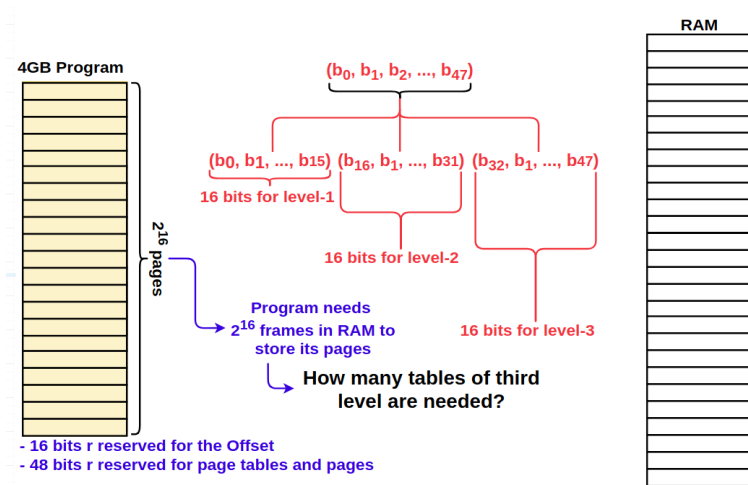
Three-level paging (Graphical example)

3. Analyze the logical address parts.



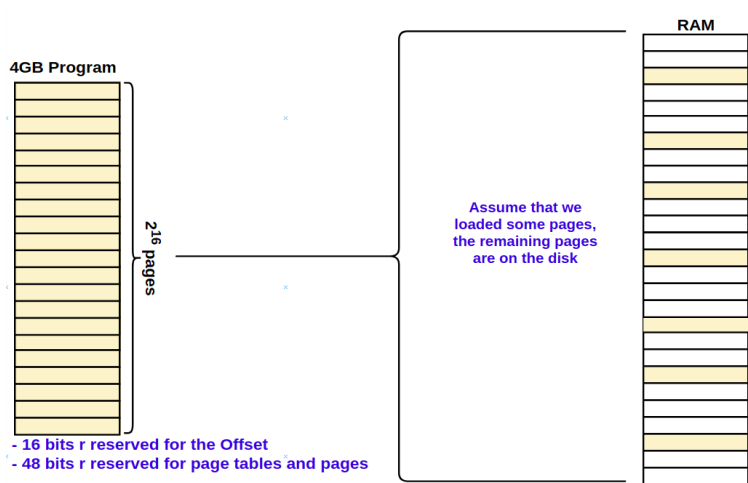
Three-level paging (Graphical example)

4. Determine the number of tables needed per level.



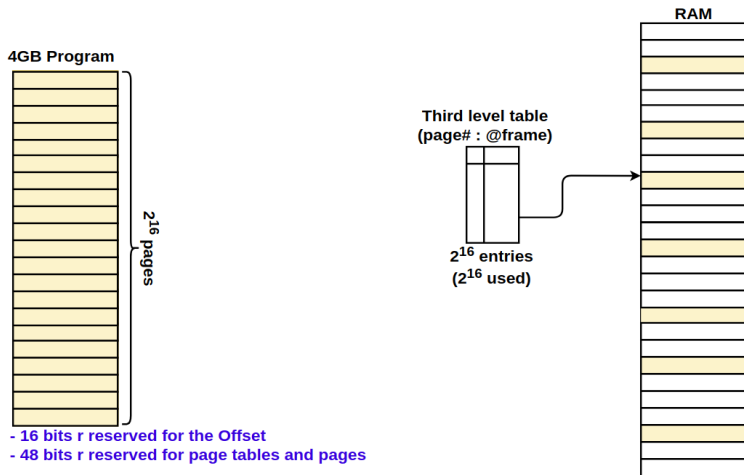
Three-level paging (Graphical example)

4. Determine the number of tables needed per level.



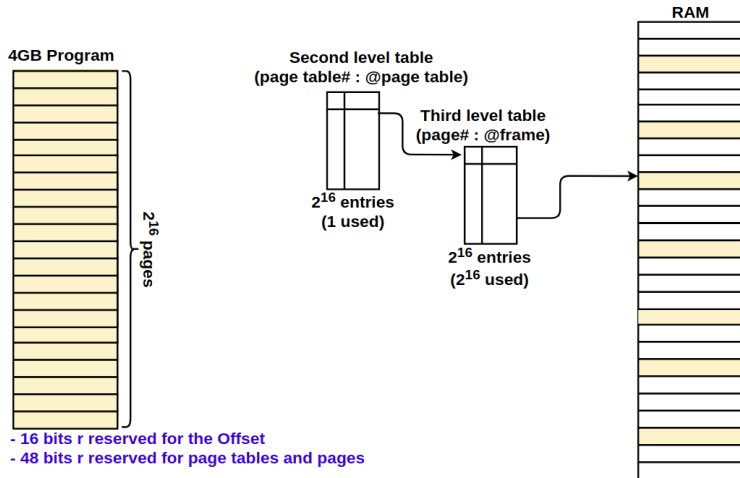
Three-level paging (Graphical example)

4. Determine the number of tables needed per level (3rd level).



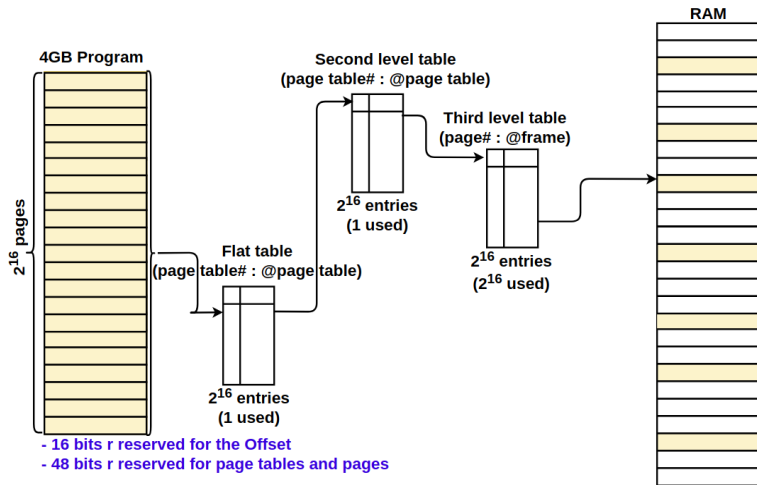
Three-level paging (Graphical example)

4. Determine the number of tables needed per level (2nd level).



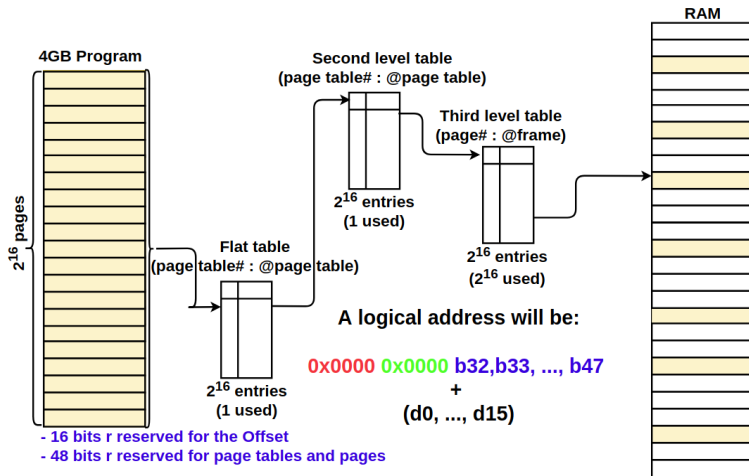
Three-level paging (Graphical example)

4. Determine the number of tables needed per level (1st level).



Three-level paging (Graphical example)

5. Check logical address translation.



Non-Contiguous Memory Allocation: Segmentation

Segmentation

Segmentation

Definition

Segmentation. Is a memory management scheme in which the user program's virtual space is divided into variable sized partitions, called segments, and stored into the memory with no predefined order.

Each segment has a name (a number) and a length.

The virtual address is expressed in terms of segment# and Offset.

Segments are constructed during the compilation phase.

e.g., a C compiler might create separate segment for the following:

- The code (text segment)
- The heap (heap segment)
- The standard C library.
- Global variables (data segment).
- The stack (stack segment).

Segmentation supports sharing (data or code segments) and memory protection as well (read/write/execute).

Segmentation

Definition

Segmentation. Is a memory management scheme in which the user program's virtual space is divided into variable sized partitions, called segments, and stored into the memory with no predefined order.

Segmentation uses a **segment table** for address translation pointed out by an STBR (Segment Table Base Register).

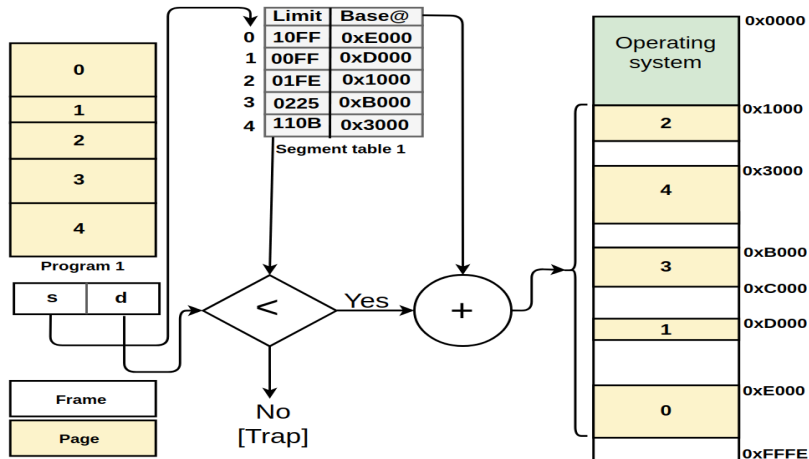
Each entry of the segment table has:

- **Segment base** that contains the base address of a given segment,
- **Segment limit** that contains the length of the segment.

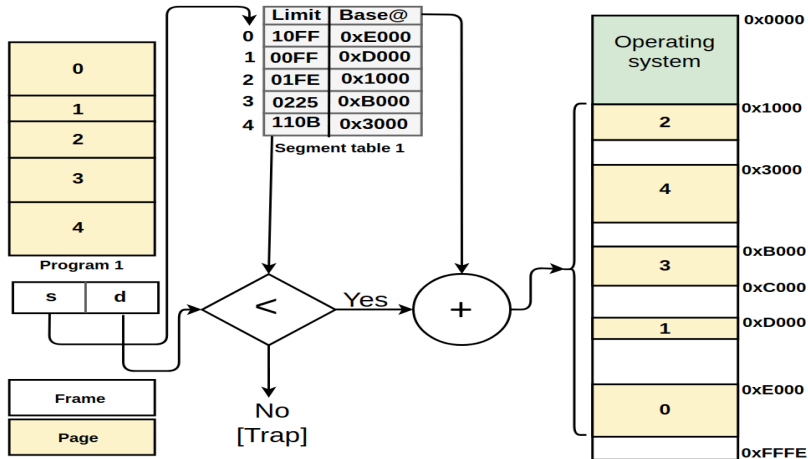
Although there is no internal fragmentation in segmentation, it suffers from external fragmentation and hence needs compaction.

Segmentation

In segmentation, the virtual address consists of a segment number s and an Offset $d \in [0\text{-Limit}[$.

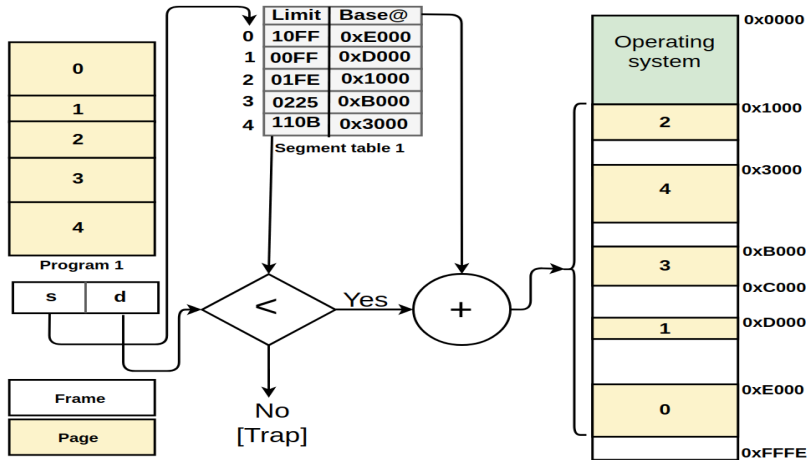


Segmentation



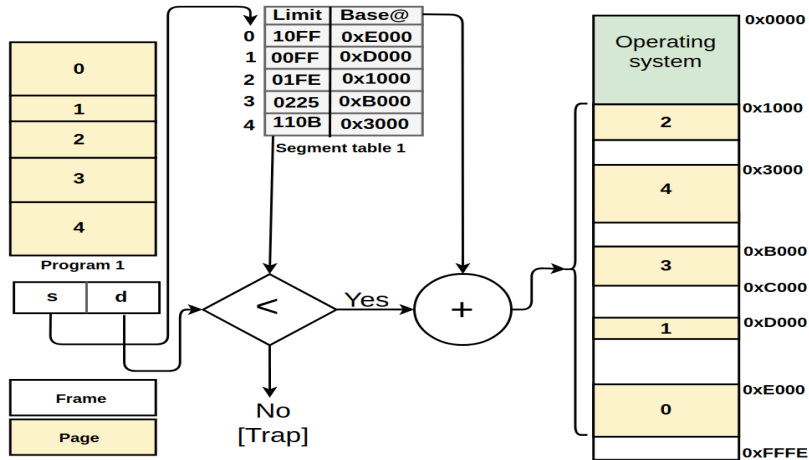
What is the physical address that corresponds to the the virtual address (4, 022B)?

Segmentation



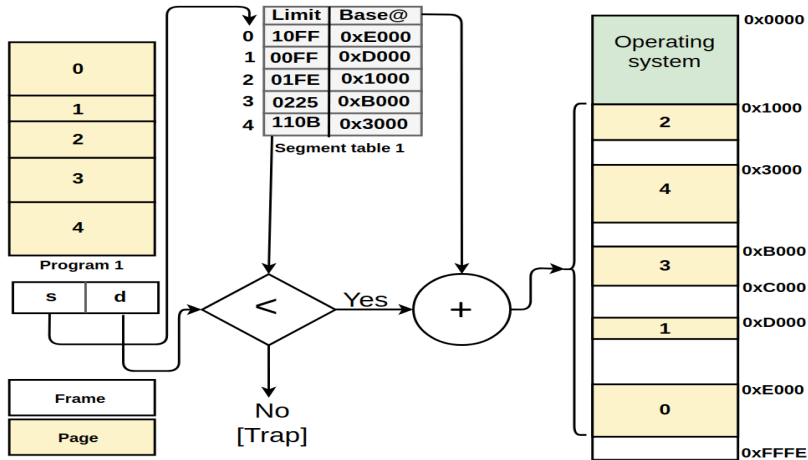
What is the physical address that corresponds to the the virtual address (**4, 022B**)? Sol: **0x322B**

Segmentation



What is the physical address that corresponds to the the virtual address (2, 0x01FF)?

Segmentation



What is the physical address that corresponds to the the virtual address (**2**, **0x01FF**)? Sol: **Trap (Segmentation Fault)**.

Segmentation with paging

Segmentation with paging

Segmentation with paging

Definition

Segmentation with paging. Is a memory management scheme in which the user program's virtual space is divided into variable sized partitions, called segments, and each segment is again divided into pages.

Each program has its own segment table pointed out by its STBR (Segment Table Base Register).

Each segment table's entry i.e., STE (Segment Table Entry), contains:

- Segment # encoded with a certain number of bits.
- Base address (physical address) of the page table for this segment.
- Segment limit.

The number of pages per segment is determined during compilation.

This memory management scheme aims to correlate the advantages of both segmentation and paging.

Segmentation with paging

In a system that uses segmentation with paging, the virtual address is expressed as:

Segment#:Segment_Offset

and interpreted as:

Segment#:Page#:Page_Offset

Example. Consider a **24-bit** computer system in which each program can have at most **256** possible segments and each segment can hold at most **256** pages of **256** bytes each.

A logical (virtual) address, e.g., $(b_0, b_1, \dots, b_{23})$ will be interpreted as:

$$[(b_0, b_1, \dots, b_{23})] : (\underbrace{b_0, b_1, \dots, b_7}_{\text{Segment Number}}, \underbrace{b_8, b_9, \dots, b_{15}}_{\text{Page Number}}, \underbrace{b_{16}, b_{17}, \dots, b_{23}}_{\text{Offset}})$$

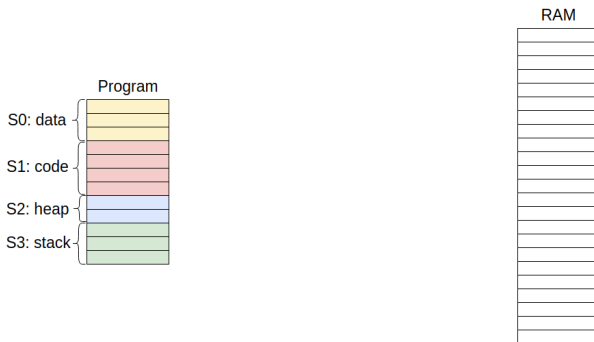
Segmentation with paging

Consider a 3KB program divided into 12 pages of 256 bytes each. The main memory is also divided into frames of 256 bytes each.



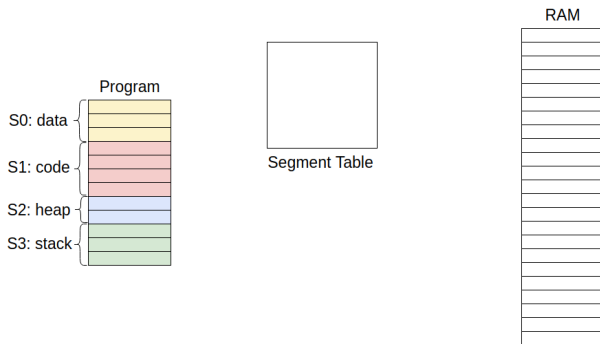
Segmentation with paging

The program is compiled and four (4) segments have been generated.



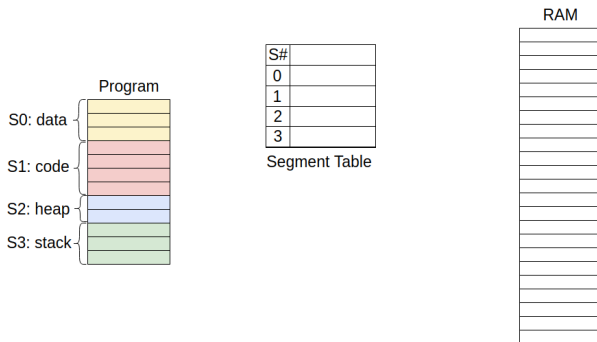
Segmentation with paging

The program will have a segment table.



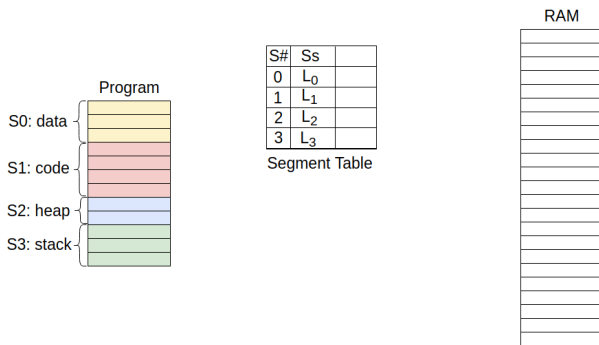
Segmentation with paging

The segment table contains: (1) A segment number.



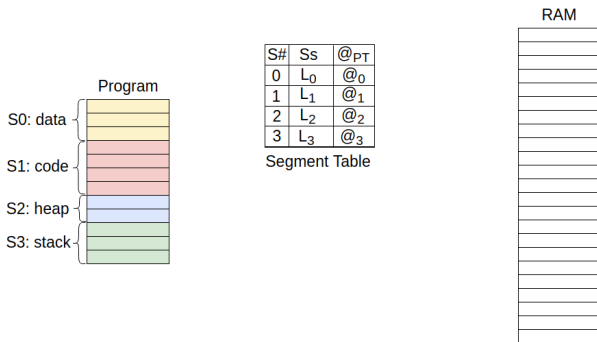
Segmentation with paging

The segment table contains: (1) A segment number (2) A segment size (Limit).



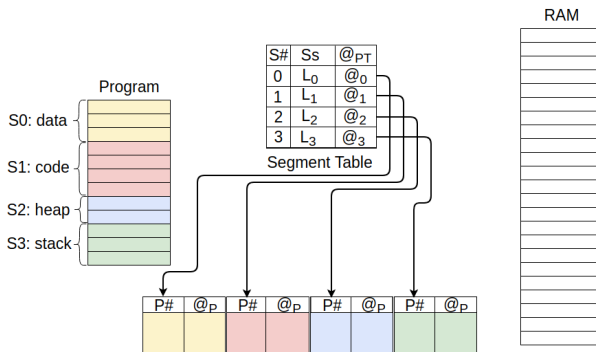
Segmentation with paging

The segment table contains: (1) A segment number (2) A segment size (Limit) (3) A base address for segment's page table.



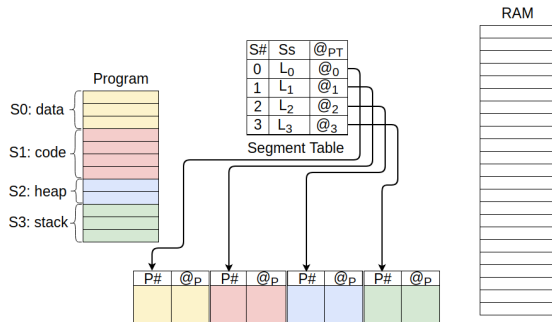
Segmentation with paging

Each base address points to a page table in the main memory.



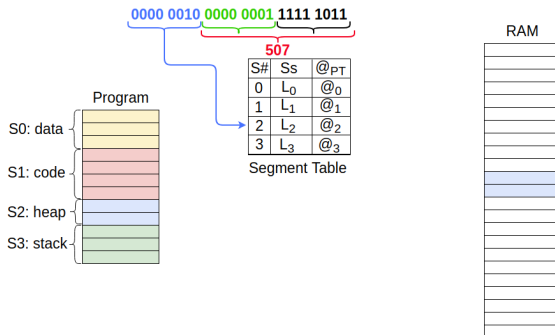
Segmentation with paging

CPU generates the following logical address: **0x0201FB**



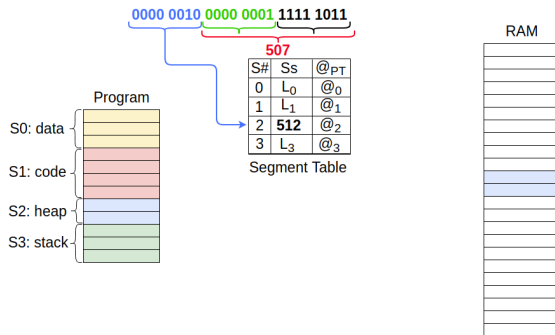
Segmentation with paging

Note that the limit is checked before address translation: **0x0201FB**



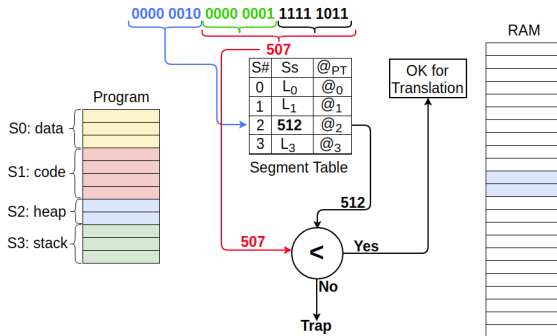
Segmentation with paging

Note that the limit is checked before address translation: **0x0201FB**



Segmentation with paging

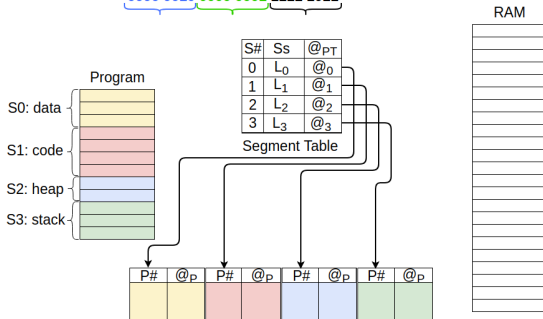
Note that the limit is checked before address translation: **0x0201FB**



Segmentation with paging

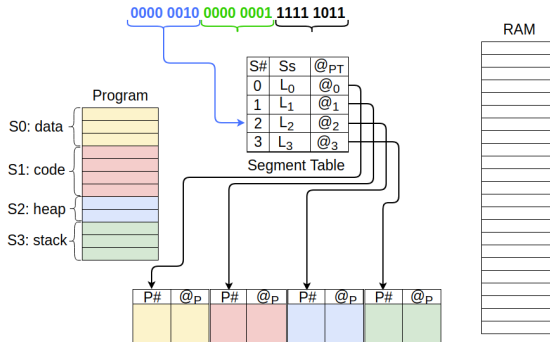
CPU generates the following logical address: **0x0201FB**

0000 0010 0000 0001 1111 1011



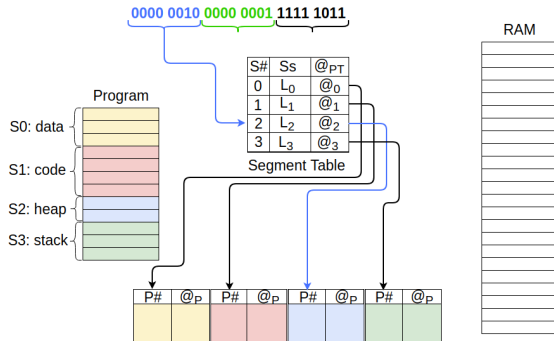
Segmentation with paging

CPU generates the following logical address: **0x0201FB**



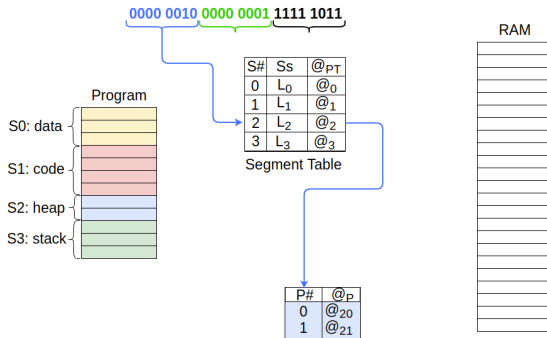
Segmentation with paging

CPU generates the following logical address: **0x0201FB**



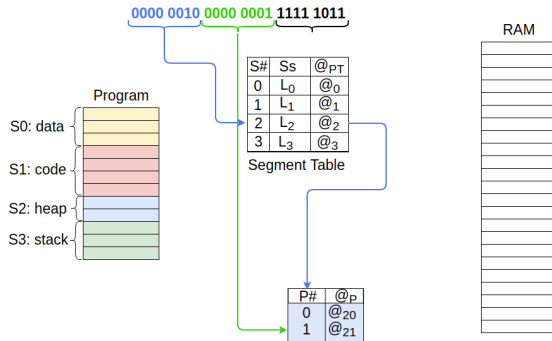
Segmentation with paging

CPU generates the following logical address: **0x0201FB**



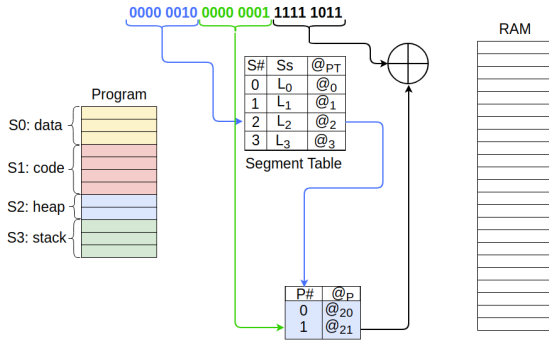
Segmentation with paging

CPU generates the following logical address: **0x0201FB**



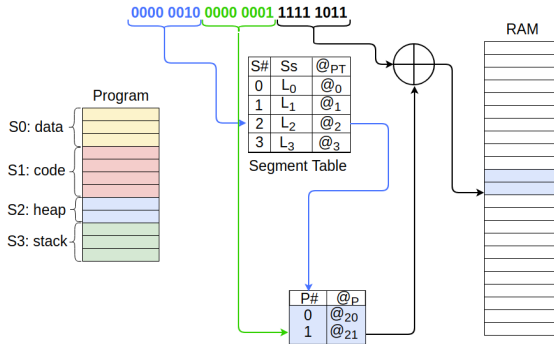
Segmentation with paging

CPU generates the following logical address: **0x0201FB**



Segmentation with paging

CPU generates the following logical address: **0x0201FB**



Segmentation with paging and TLB

The TLB (Translation Look-aside Buffer) keeps entries of the form:

Segment# : Page# : Limit : Frame#

- A TLB hit produces a Frame# (base address of the frame in RAM).
- A TLB mis:
 - 1 CPU accesses segment table using STBR.
 - 2 CPU checks the offset with the limit:
If (segment offset $\geq$ limit) { trap; } else { goto (3); }
 - 3 CPU reads the base address of the page table.
 - 4 CPU accesses page table.
 - 5 CPU fetches page using page#, then:
If (page is on the disk) { raise(Page fault**); } else { goto (6); }**
 - 6 CPU reads the base address of the fetched page using page#.
 - 7 CPU accesses the corresponding frame and use offset to read the Byte.

Segmentation with paging

Definition

Page fault. Is an exception that occurs when the CPU tries to access a page that is not present in the main memory.

In case of a page fault:

- 1 The current program is interrupted and page fault handler is invoked.
- 2 The handler initiates a disk transfer to load the page into memory.

The memory management module allocates a frame to host the new page. If no frame is available, the memory management module swaps out a “victim” page from the memory following a page replacement algorithm.

- 3 Once done, the CPU is informed (e.g., DMA completion interrupt).
- 4 Interrupt handlers updates the page table and TLB.
- 5 The CPU restarts the instruction of the interrupted program.

Segmentation with paging

Definition

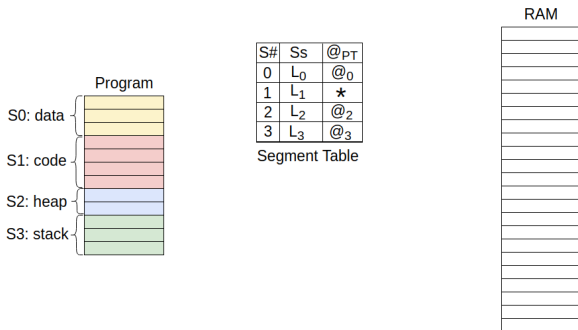
Segment fault. Is an exception that occurs when the CPU tries to access a segment that is not present in the main memory.

In case of a segment fault:

- 1 The current program is interrupted and segment fault handler is invoked.
- 2 The handler initiates a disk transfer to load the page table of that segment.
i.e., if a page table can be hosted by one frame, then a frame is allocated to host the new page table.
- 3 Once done, the CPU is informed and the segment table is updated.
- 4 Try access the segment. Then if the requested page is not in memory, a page fault is raised then handled.
- 5 TLB is updated as well.

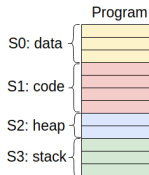
Segmentation with paging

Let us consider the previous example, where a 3KB program is divided into 12 pages of 256 bytes each. And let Segment 1 be stored on the disk (*).



Segmentation with paging

CPU generates the following logical address: **0x01028C**



S#	Ss	@PT
0	L ₀	@ ₀
1	L ₁	*
2	L ₂	@ ₂
3	L ₃	@ ₃

Segment Table

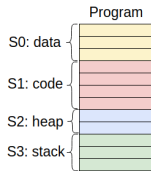
RAM



Segmentation with paging

CPU generates the following logical address: **0x01028C**

0000 00010000 00101000 1100



S#	Ss	@ _{P_T}
0	L ₀	@ ₀
1	L ₁	*
2	L ₂	@ ₂
3	L ₃	@ ₃

Segment Table

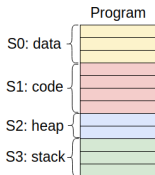
RAM



Segmentation with paging

CPU generates the following logical address: **0x01028C**

0000 0001 0000 0010 1000 1100



S#	Ss	@ _P T
0	L ₀	@ ₀
1	L ₁	*
2	L ₂	@ ₂
3	L ₃	@ ₃

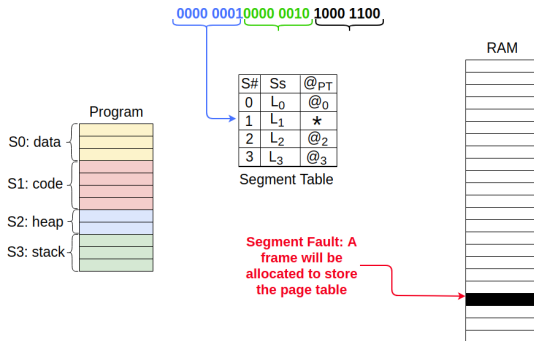
Segment Table

RAM



Segmentation with paging

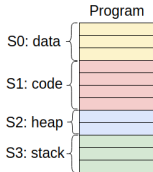
CPU generates the following logical address: **0x01028C**



Segmentation with paging

CPU generates the following logical address: **0x01028C**

0000 0001 0000 0010 1000 1100



S#	SS	@ _{PT}
0	L ₀	@ ₀
1	L ₁	*
2	L ₂	@ ₂
3	L ₃	@ ₃

Segment Table

P#	@ _P
0	*
1	*
2	*
3	*

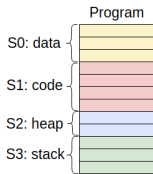
RAM



Segmentation with paging

CPU generates the following logical address: **0x01028C**

0000 0001 0000 0010 1000 1100

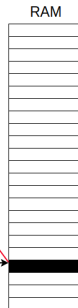


S#	Ss	@p _T
0	L ₀	@ ₀
1	L ₁	@ ₁
2	L ₂	@ ₂
3	L ₃	@ ₃

Segment Table

P#	@p
0	*
1	*
2	*
3	*

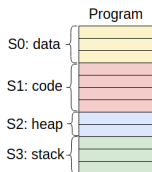
Update
Segment
Table



Segmentation with paging

CPU generates the following logical address: **0x01028C**

0000 0001 **0000 0010** **1000 1100**



S#	Ss	@ _{PT}
0	L ₀	@ ₀
1	L ₁	@ ₁
2	L ₂	@ ₂
3	L ₃	@ ₃

Segment Table

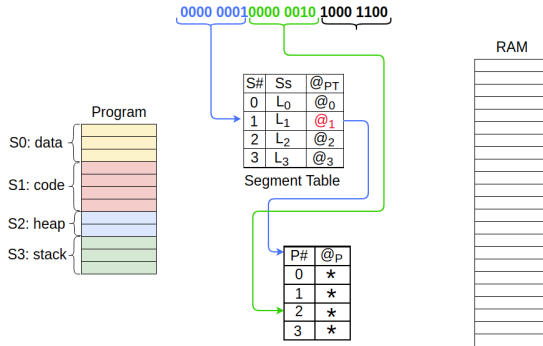
P#	@ _P
0	*
1	*
2	*
3	*

RAM



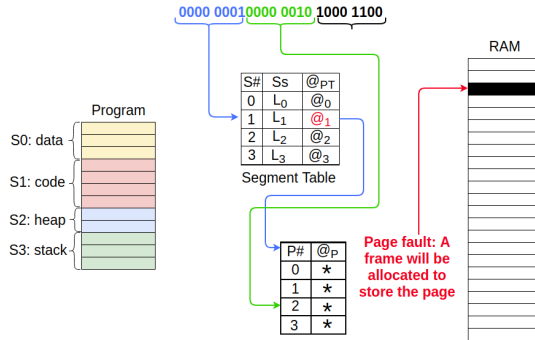
Segmentation with paging

CPU generates the following logical address: **0x01028C**



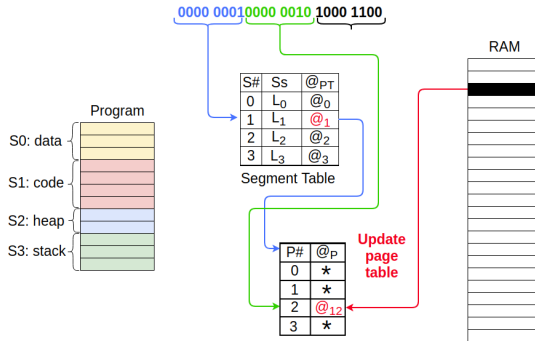
Segmentation with paging

CPU generates the following logical address: **0x01028C**



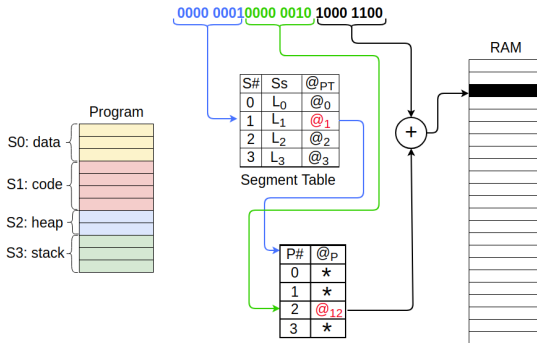
Segmentation with paging

CPU generates the following logical address: **0x01028C**



Segmentation with paging

CPU generates the following logical address: **0x01028C**



Segmentation with paging

Exercise:

STBR=0x06

List of free page frames: 0x1A, 0x08, 0x21, 0x16, 0x0E.

Segment tables:

0x0200	0x0F
	0x13
	0x09
	*
	4E

0x0600	0x09
	0x0A
	0x4C
	0x3D
	0x5D

0x3000	0x0F
	0x09
	0x13
	0x4C
	*

Page tables:

0x3D00	0x5C
	0x1E
	0x5A
	59
	*

0x4C00	*
	0x3B
	*
	0X20
	*

0x0A00	*
	0x57
	*
	*
	*

0x0900	04
--------	----

0x0F00	*
	0x25

0x1300	0x18
--------	------

0x4E00	11
	*

What is the physical address of: 0x030260, 0x0101F0, 0x0102AA?

Virtual Memory

Memory Management: Virtual Memory

Virtual Memory

Definition

Virtual Memory. Is a memory management technology in which programs can be executed without being completely loaded into memory.

It makes use of both main memory and secondary storage device (e.g., hard disk drive or solid state drive) to give the illusion of **unlimited memory**.

Programs virtual space is divided into pages. Some pages are loaded, whereas others are left on the secondary storage device (use **Demand Paging technology**).

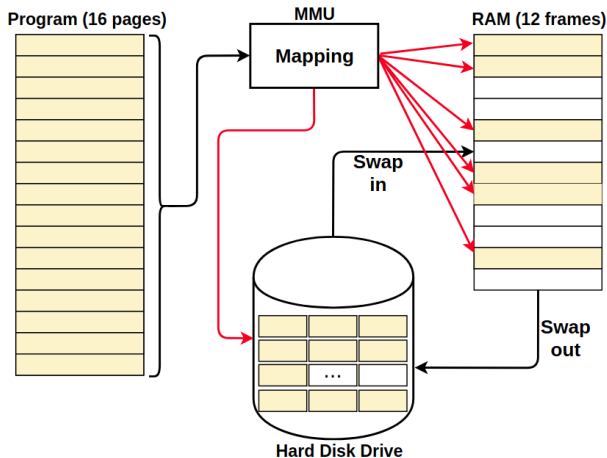
The frequently used or requested pages are loaded (swapped in), whereas the less used ones are swapped out to the disk (**some may never be loaded**).

An examination of real programs shows that, in many cases, the entire program is not needed, e.g., **Code handling errors, choice statements, event-driven, etc.**

It allows programs that have a size which is larger than the available physical memory to be executed, e.g., **execute a 16GB program on 4GB RAM.**

Separate the logical memory (perceived by user) from physical memory

Program Execution with Virtual Memory



The minimum amount of physical RAM (# of memory frames) a program needs to be actively resident in memory is known as the **working-set**.

Demand Paging

Definition

Demand paging. Is a techniques used in virtual memory, consisting of loading pages only as they are needed during program execution.

- Pages that are never accessed, are never loaded into main memory.
- **Pure demand paging.** A program starts execution with no pages in memory (i.e., never bring a page into RAM till it is required).
- We use the validity bit to determine whether a page is in the memory or not. If the bit is **valid**, then the page is both legal and in the memory. Otherwise, the page is either is not legal or is valid but in the HDD.
- As long as the process accesses pages that are in the memory, the execution proceeds normally. When a process tries to access a page that is either invalid or not in the memory, a **page fault** is raised.

Handling the page fault

When the page fault is raised, an interrupt is generated, and the page fault handler is invoked:

- 1 The handler checks whether the cause of the page fault is due to illegal access (e.g., **segmentation fault**) or a missing page.

If it was because of illegal access, the process is terminated.
Otherwise, the requested page should be swapped in.

- 2 The handler allocates a frame in the RAM to host the new page.
- 3 A disk transfer is initiated to read the desired page into the memory.
- 4 Once done, the handler updates the pages table.
- 5 The interrupted process is **restarted** from the last instruction.

Performance in Demand paging scheme

For most computers, the memory-access time, ranges from 10 to 200ns.

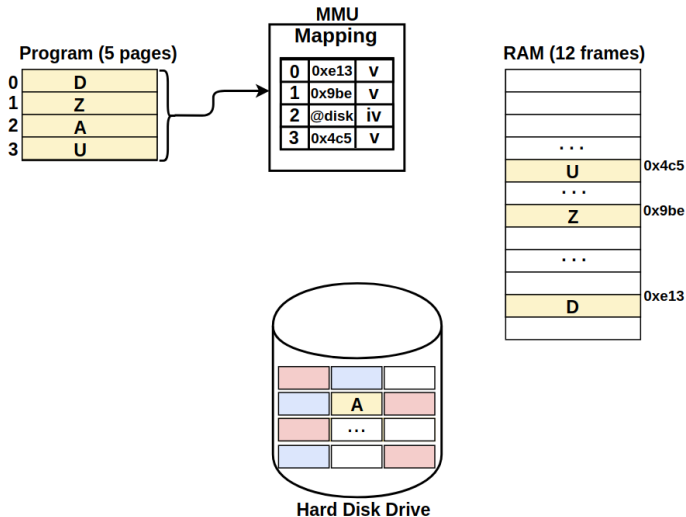
As long as we don't have page faults: the effective access time is equal to the memory access time.

But when page fault happens, the effective access time changes: *Let p be the probability of a page fault, and $t_{page-fault}$ be the time for page fault handling, and $t_{memory-access}$ the memory-access time. Then, the effective access time t is:*

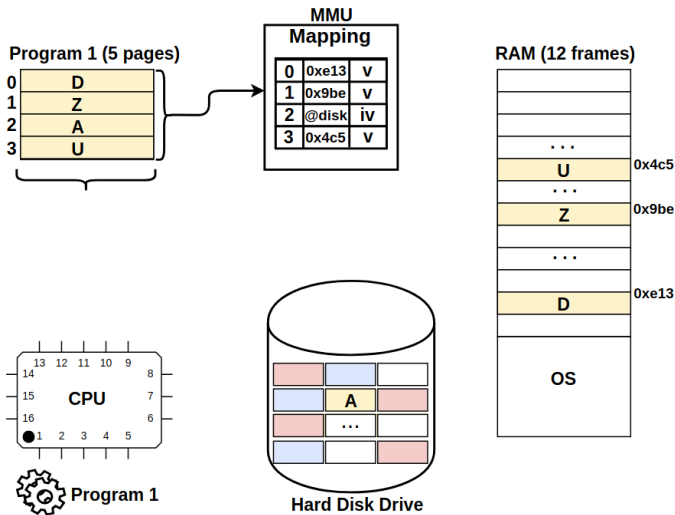
$$t = [(1 - p) \times t_{memory-access}] + [p \times t_{page-fault}]$$

What is the amount of time of $t_{page-fault}$?

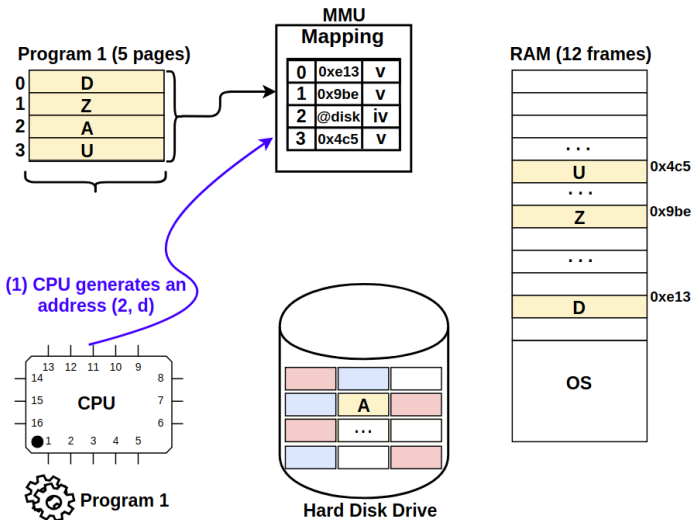
Page fault handler



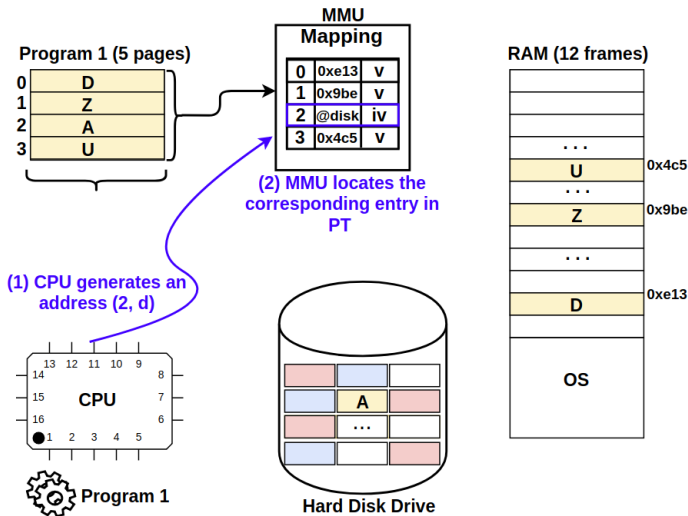
Page fault handler



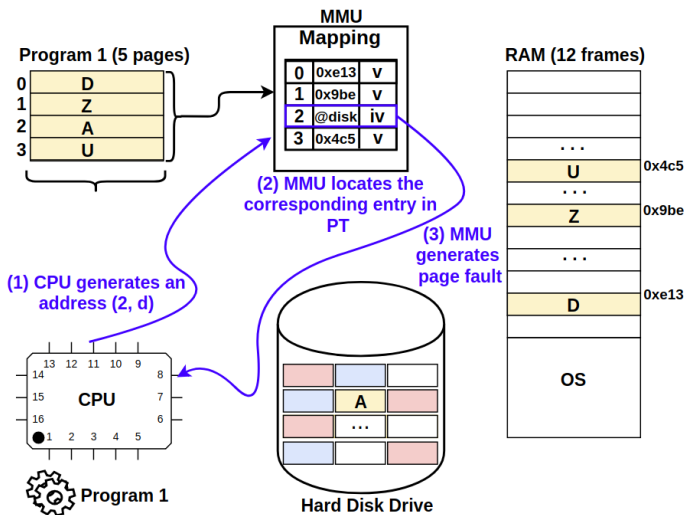
Page fault handler



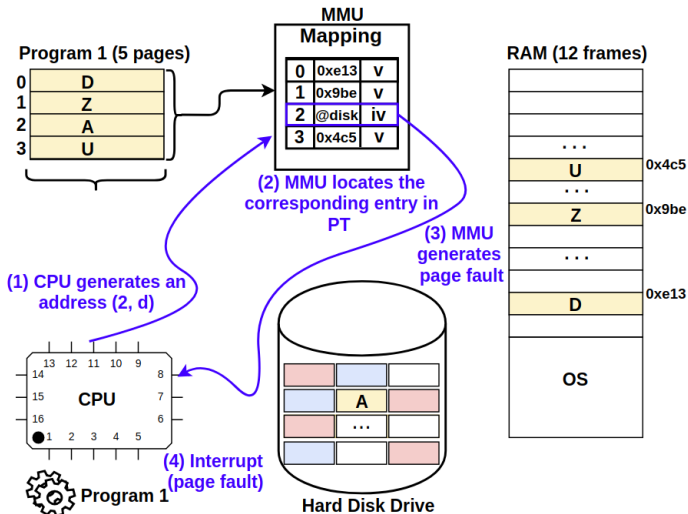
Page fault handler



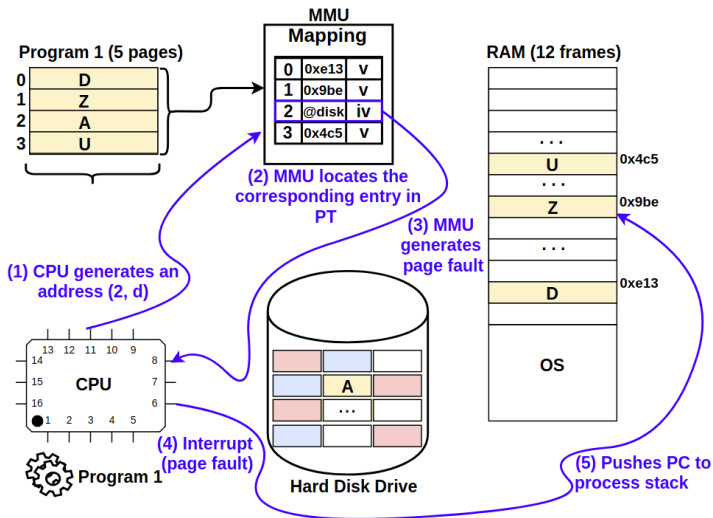
Page fault handler



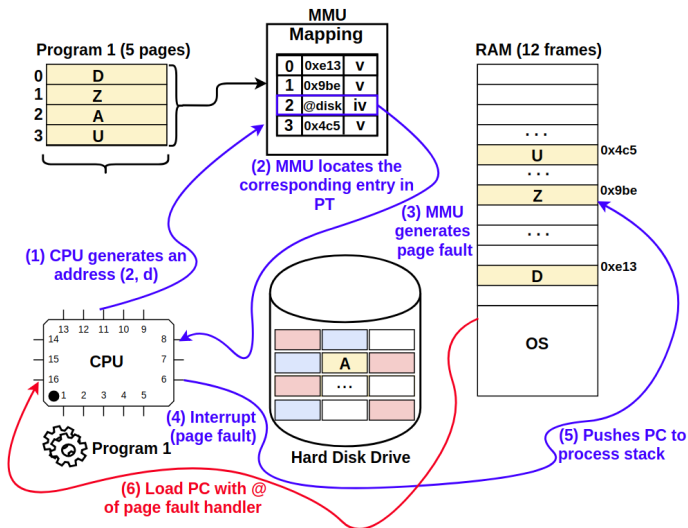
Page fault handler



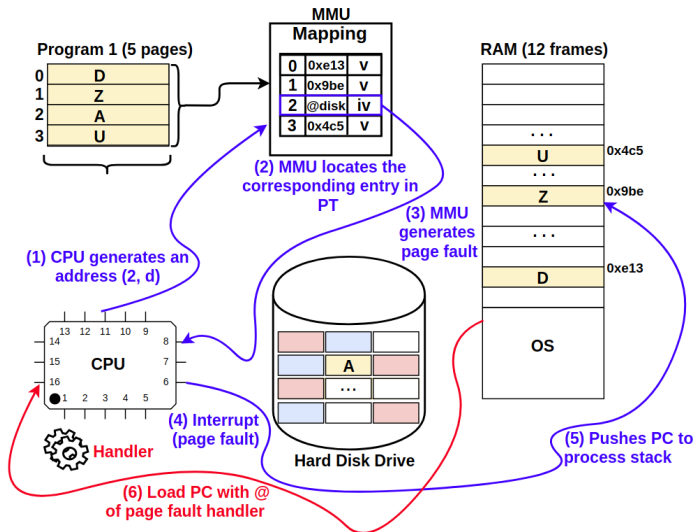
Page fault handler



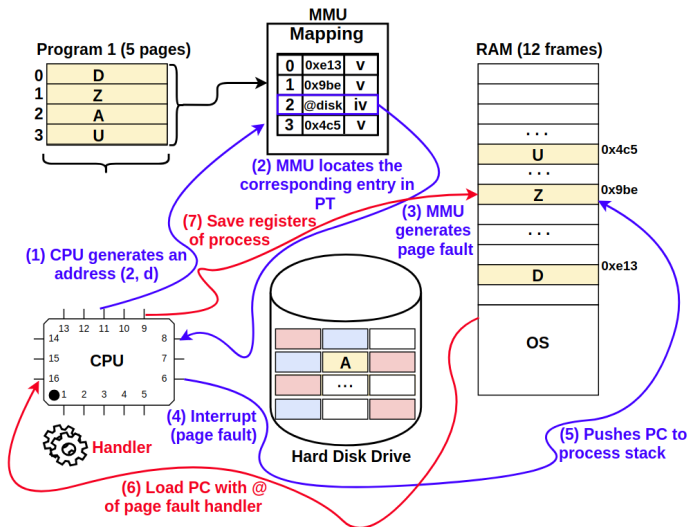
Page fault handler



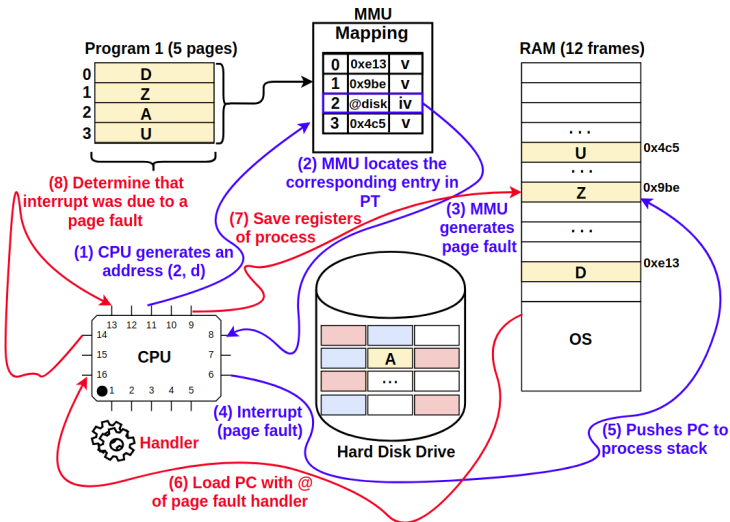
Page fault handler



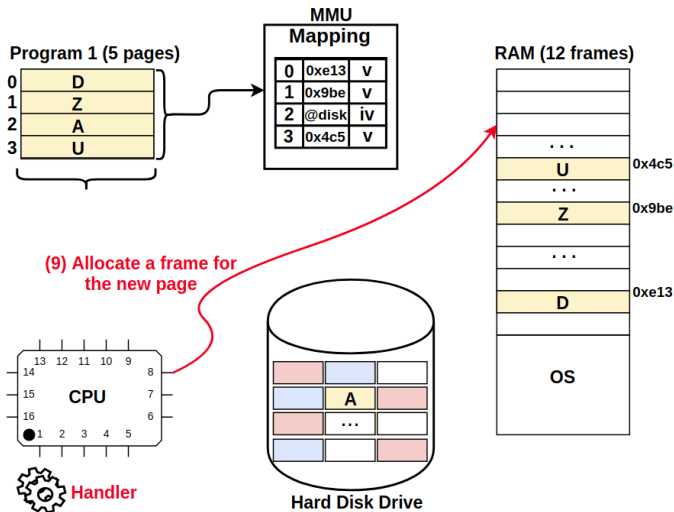
Page fault handler



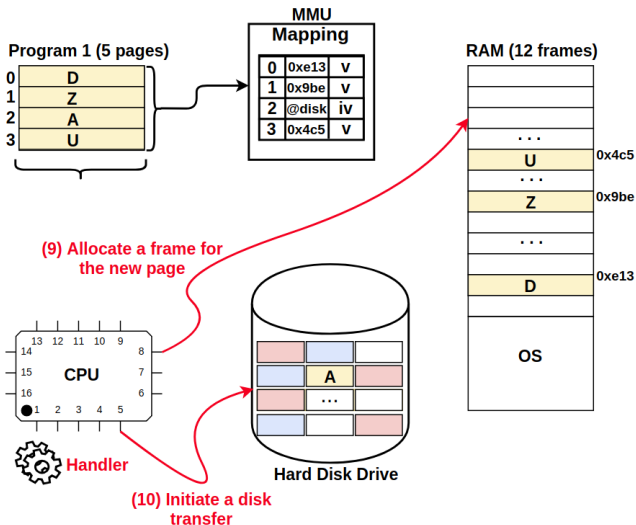
Page fault handler



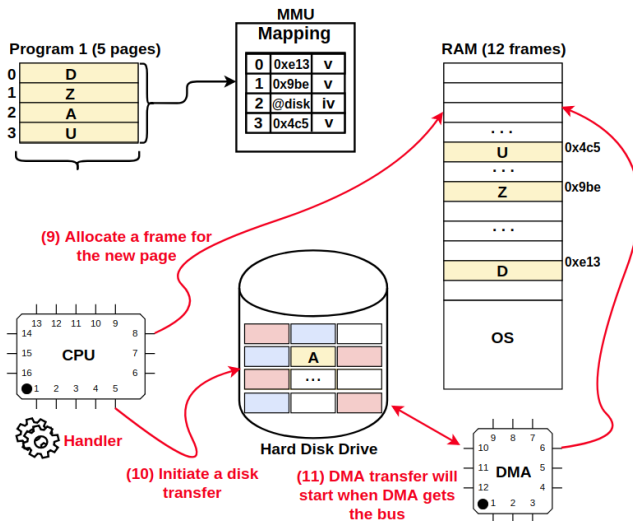
Page fault handler



Page fault handler



Page fault handler

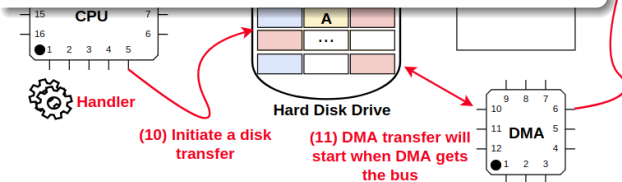


Page fault handler

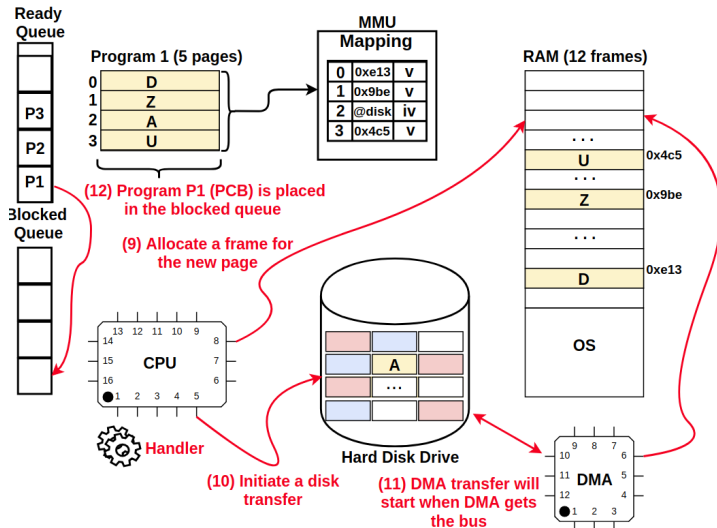
MMU Mapping

Since a DMA transfer has been initiated to resolve the cause of the interrupt (page fault), the concerned process should be context-switched by another one (it becomes a blocked process).

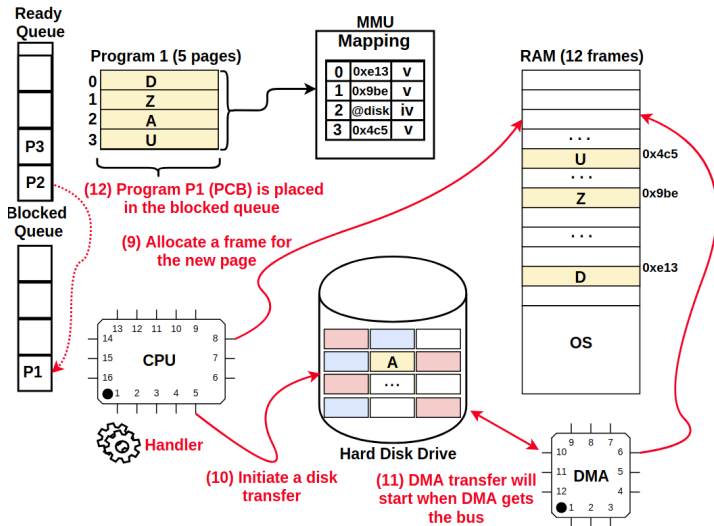
The process registers (saved on the stack) will be saved into the PCB which will get placed on the back of another queue (blocked queue).



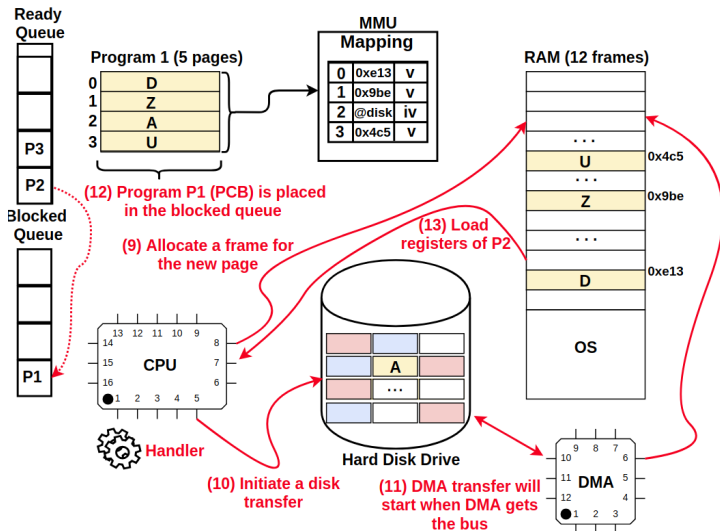
Page fault handler



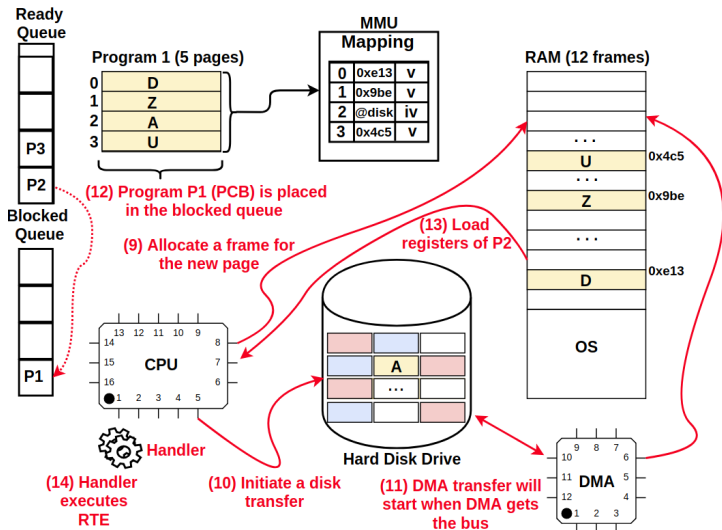
Page fault handler



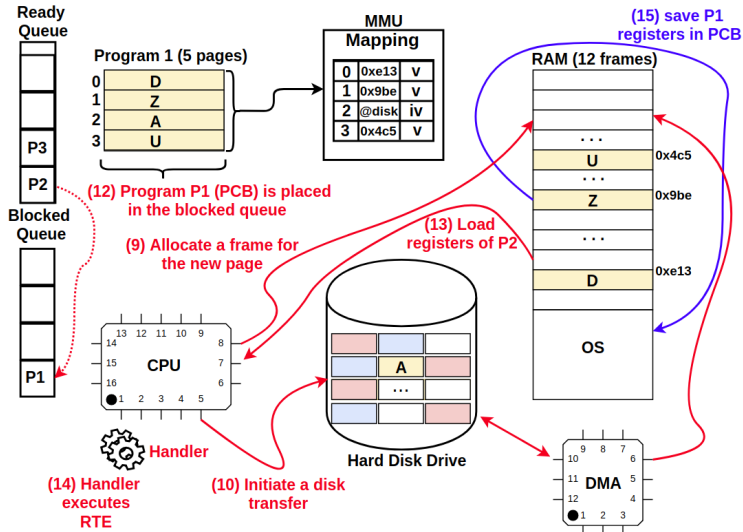
Page fault handler



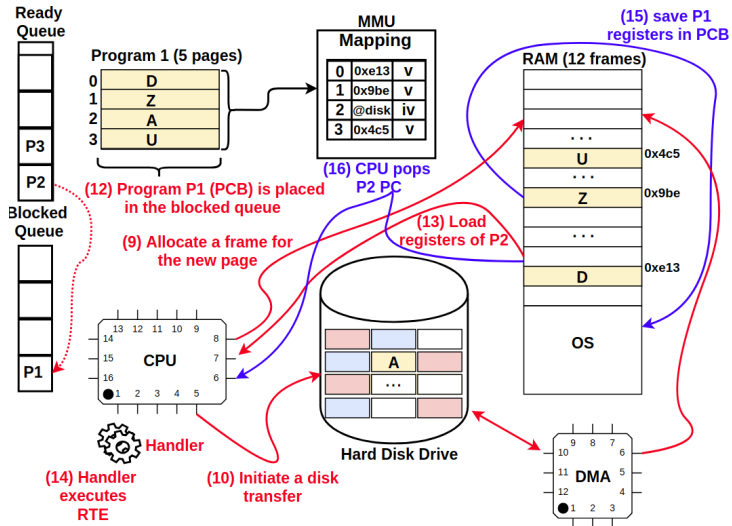
Page fault handler



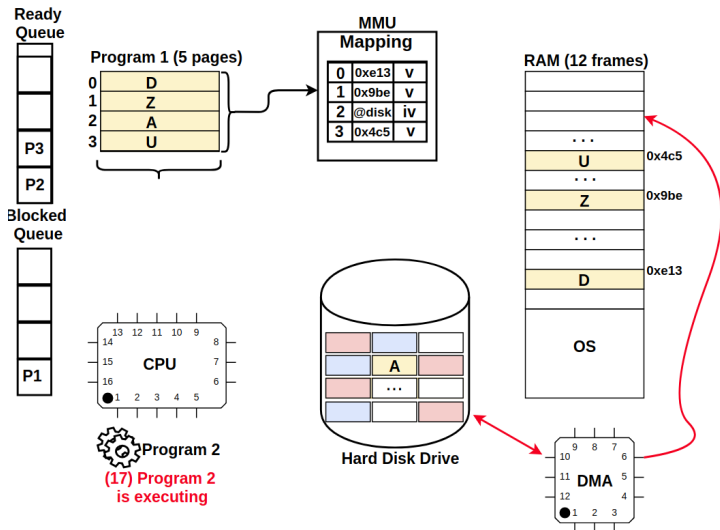
Page fault handler



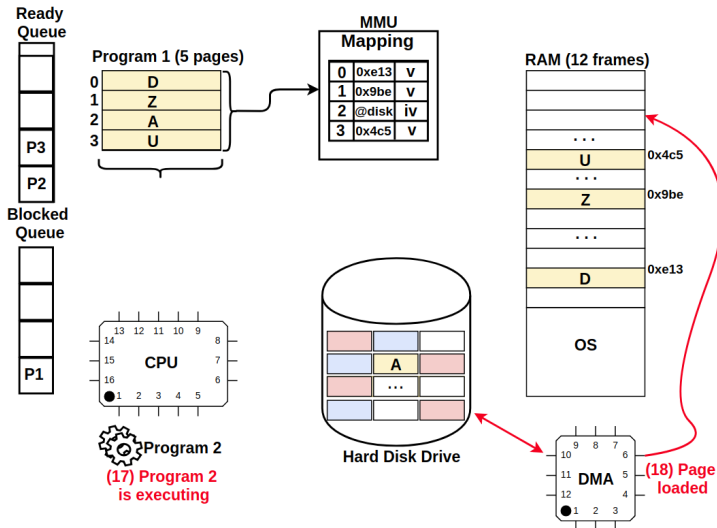
Page fault handler



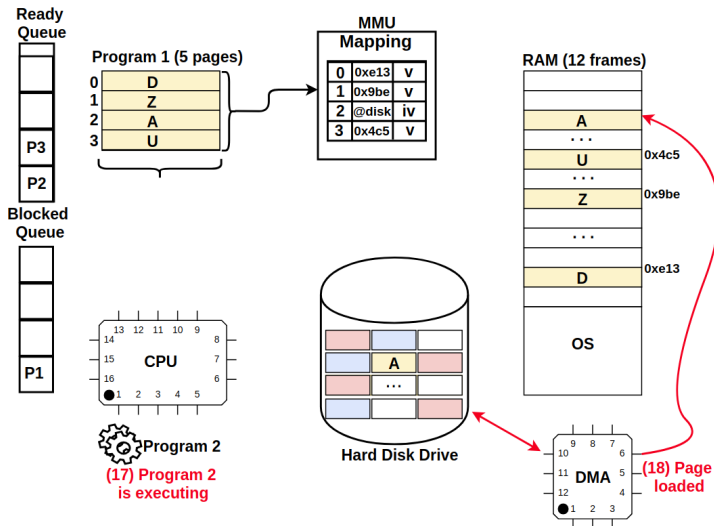
Page fault handler



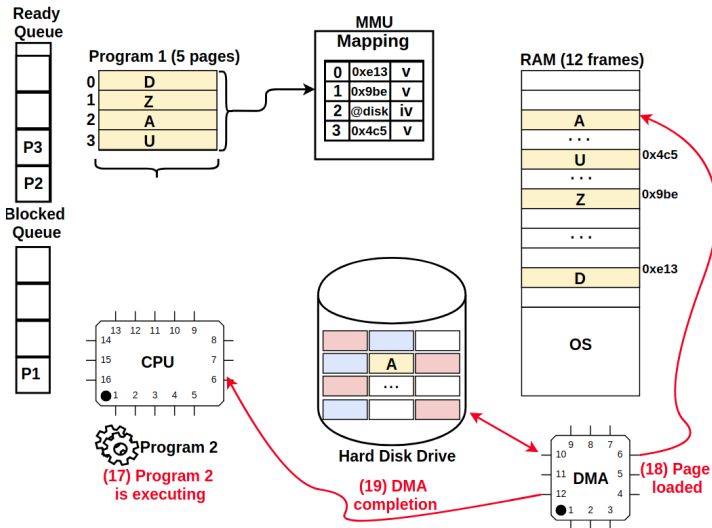
Page fault handler



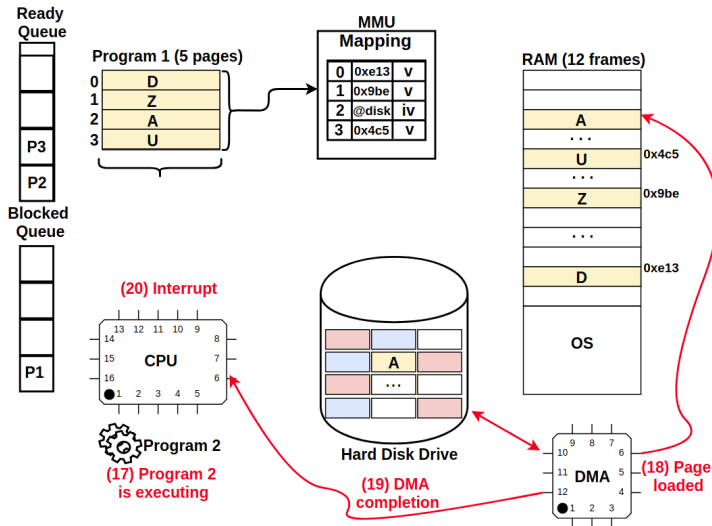
Page fault handler



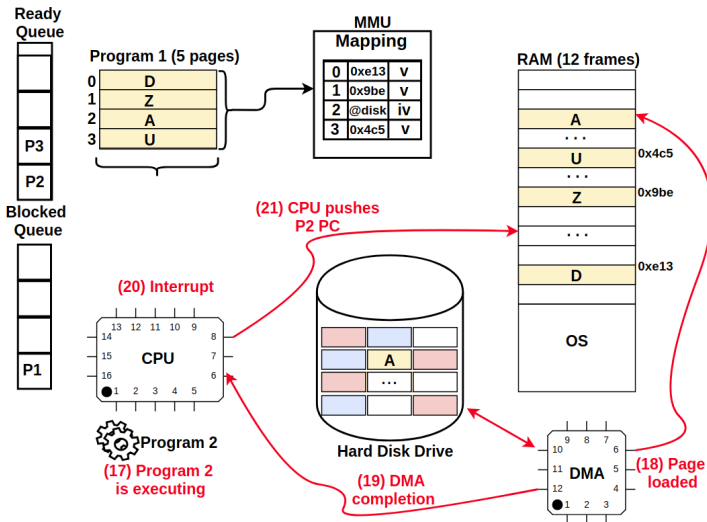
Page fault handler



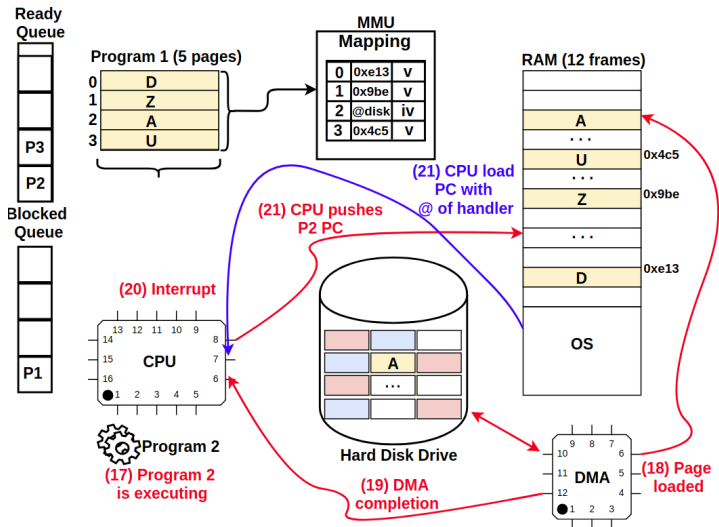
Page fault handler



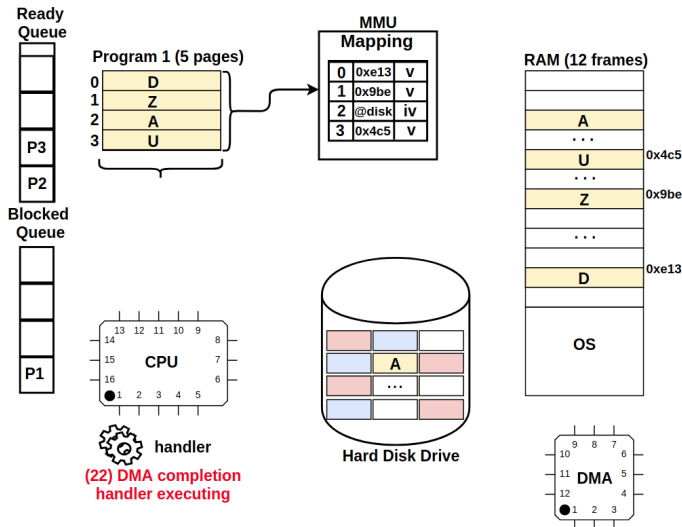
Page fault handler



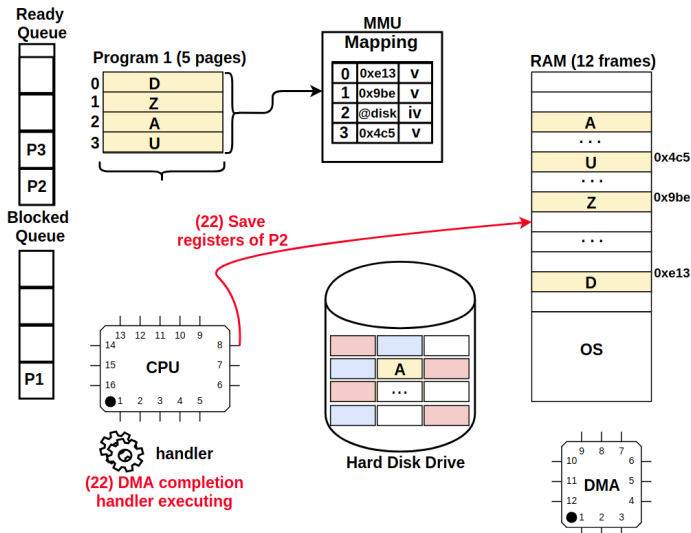
Page fault handler



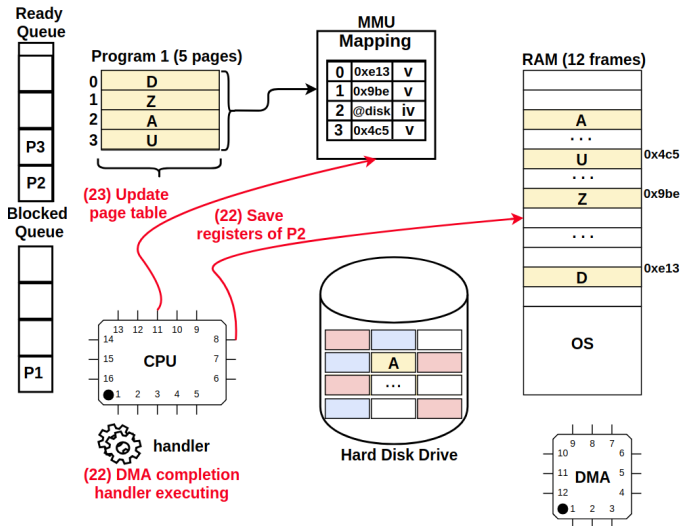
Page fault handler



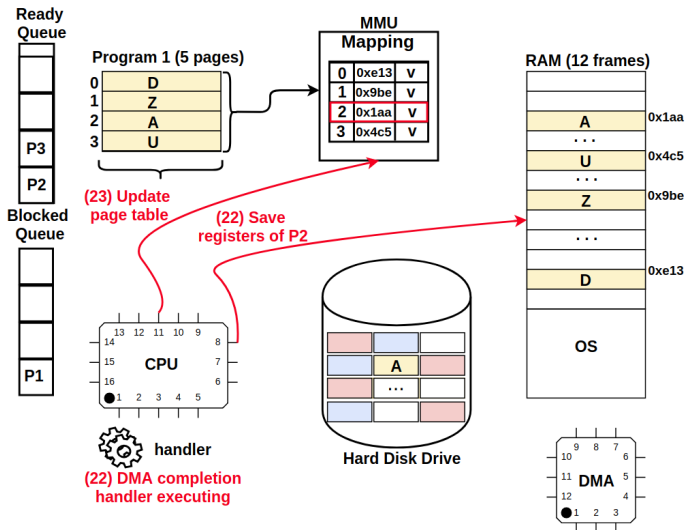
Page fault handler



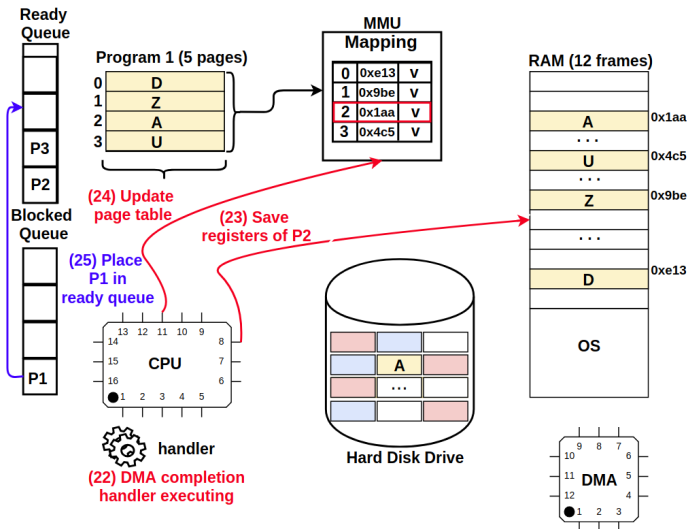
Page fault handler



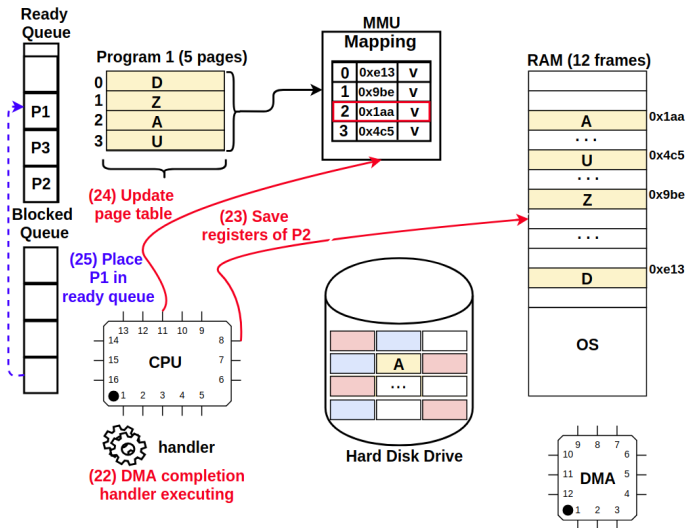
Page fault handler



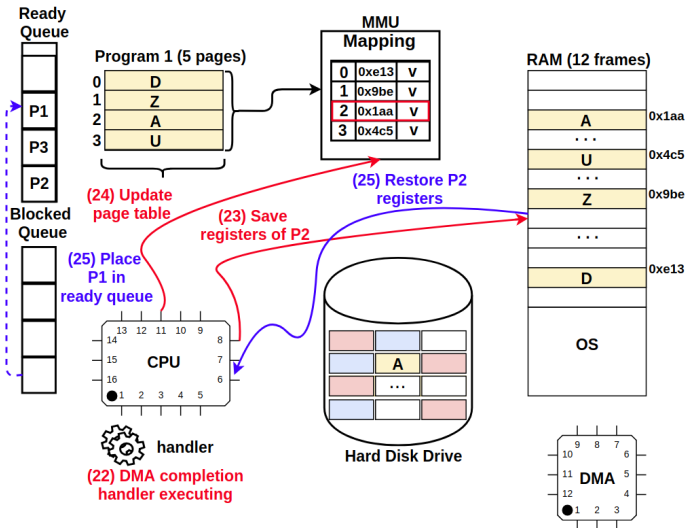
Page fault handler



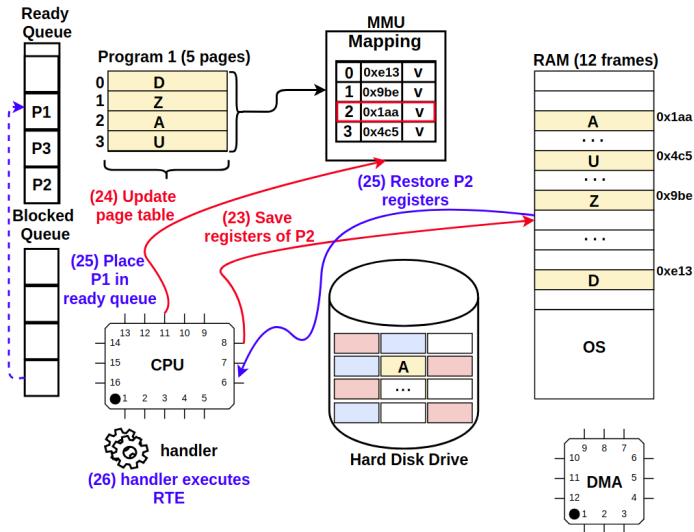
Page fault handler



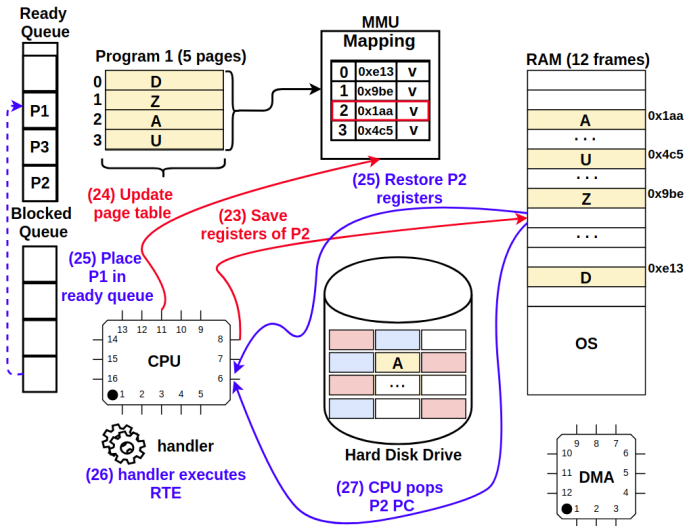
Page fault handler



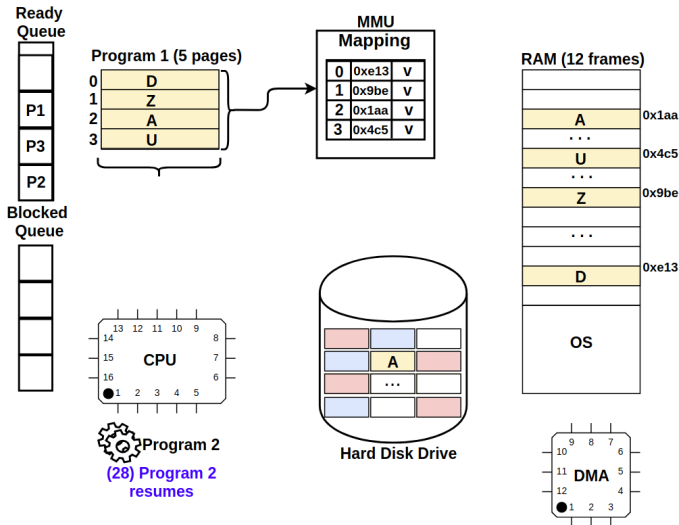
Page fault handler



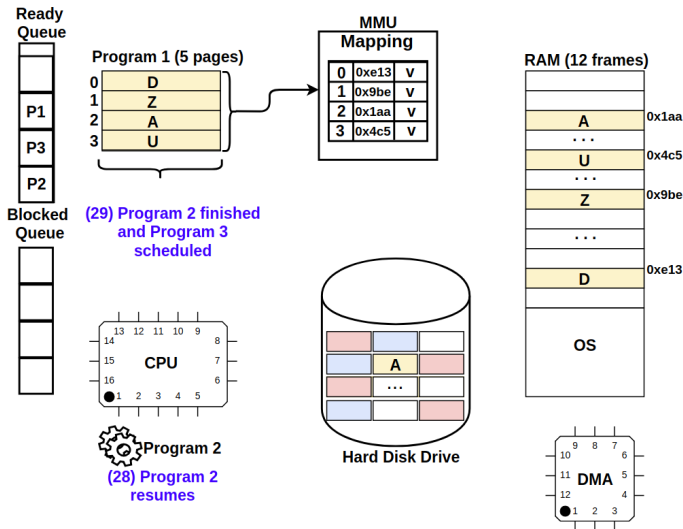
Page fault handler



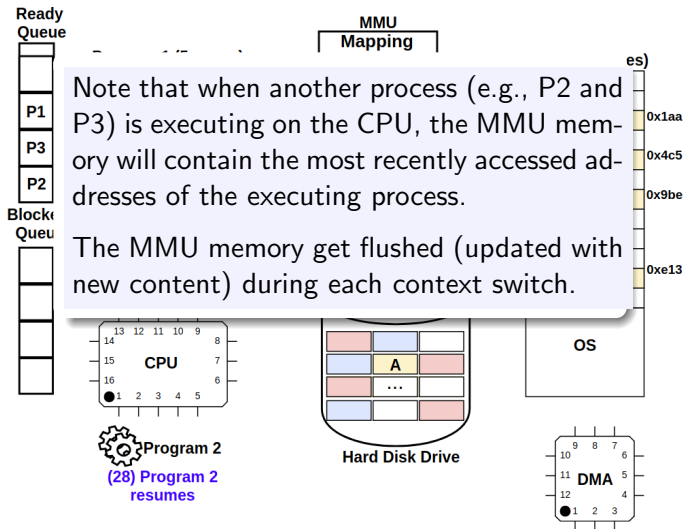
Page fault handler



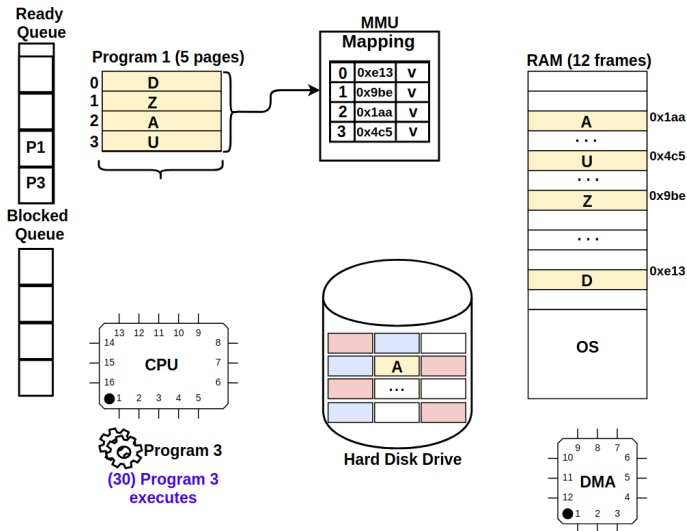
Page fault handler



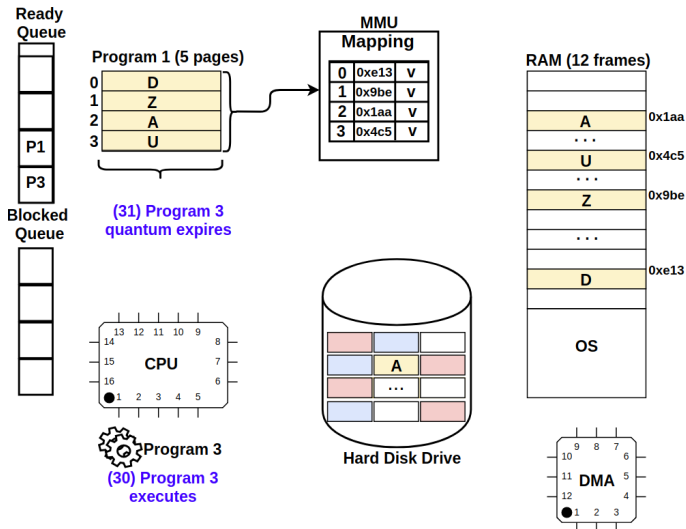
Page fault handler



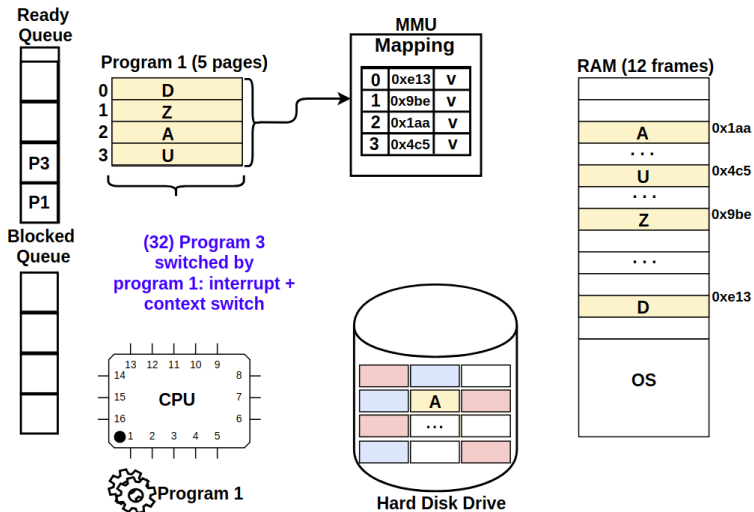
Page fault handler



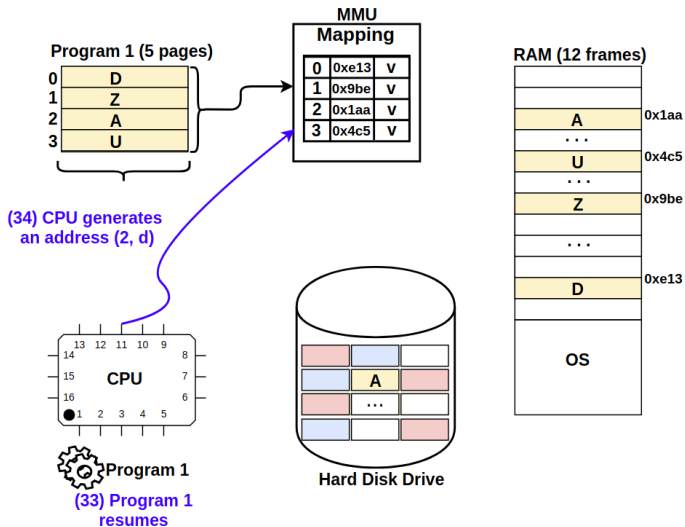
Page fault handler



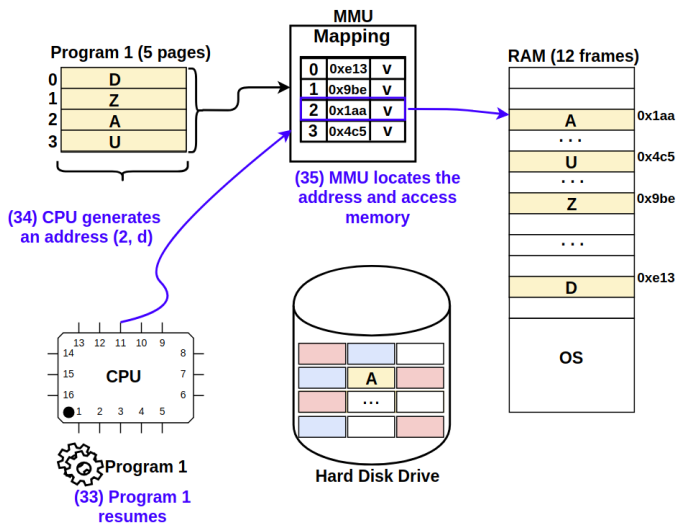
Page fault handler



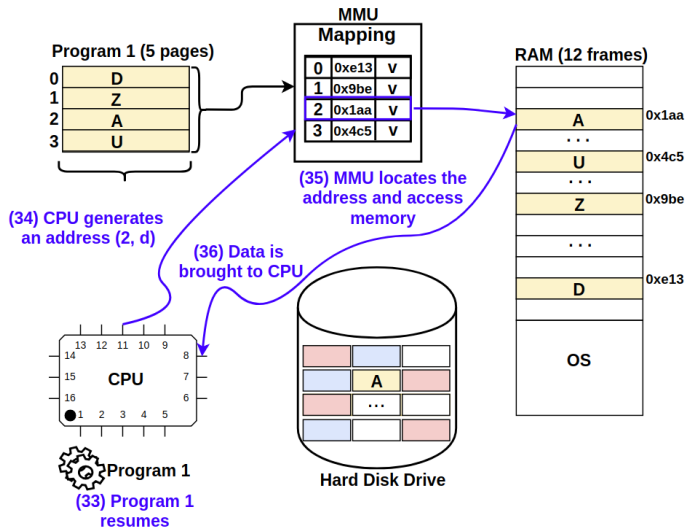
Page fault handler



Page fault handler



Page fault handler



Page Replacement

Page Replacement

Page replacement

Definition

Page replacement. Is a mechanism that allows the OS to select a page among many in the RAM and swap it out to HDD to free a frame to host a new page from the HDD.

When a page fault occurs, the OS allocates a frame in RAM to host the new page.

Yet, it could happen that there are no free frames available in the RAM.

The OS has several options at this point:

- Terminate the process (not the best option) — used by old mobile OSs.
- Swap out a victim page or an entire process (i.e., a group of frames).

A page is swapped out if its **modify bit** (or **dirty bit**) is set to 1.

Page replacement

To free a page frame in the main memory, the OS has to select a victim frame, and swap it out of the memory to the disk (if dirty).

Page tables update. As a consequence of swapping out a page and swapping in another one, both page tables have to be updated:

- (1) The page table of the process which caused the page fault and
- (2) The page table of the process whose page has been swapped out.

Page-fault handler. Does the following:

- Find the location of the desired page on the disk.
- Find a free frame:
 - If there is a frame, use it. Otherwise, run a page replacement algorithm to select a victim frame.
 - Write the victim frame to the disk (if dirty) & update the page table.
- Load the desired page into the newly freed frame and update the page table.
- Continue the interrupted process.

Page replacement

If the victim page was dirty, it should be written back on the disk. This doubles the page fault service time and increases the effective access time accordingly.

Every operating system has its own page replacement algorithm. The best algorithm is the one which has the **lowest page-fault rate**.

We can evaluate a page replacement algorithm by running it on a particular string of memory references, known as **reference string**, and computing the number of page faults.

A reference string can be generated: (1) Artificially, using a random number generator, or (2) Trace the accessed memory addresses.

We just need to consider the page# and ignore the immediate accesses to the same page.

Page replacement

Example. If we trace a particular process, where the logical addresses are on **16 bites**, we can record the following address sequence:

0x0100, 0x0432, 0x0101, 0x0612, 0x0102, 0x0103, 0x0104,
0x0101, 0x0611, 0x0102, 0x0103, 0x0104, 0x0101, 0x0610,
0x0102, 0x0103, 0x0104, 0x0101, 0x0609, 0x0102, 0x0105.

Considering a page size of **256 Bytes**. The sequence is simplified to:

0x01, 0x04, 0x01, 0x06, 0x01, 0x01, 0x01, 0x01, 0x06, 0x01,
0x01, 0x01, 0x01, 0x06, 0x01, 0x01, 0x01, 0x01, 0x06, 0x01,
0x01.

Then, by ignoring the immediate access of a page that has previous been accessed, we construct the reference sequence:

1, 4, 1, 6, 1, 6, 1, 6, 1, 6, 1.

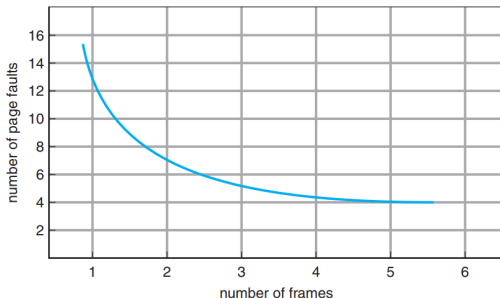
Page replacement

Now, to determine the number of page faults for a particular reference string and page replacement algorithm:

1, 4, 1, 6, 1, 6, 1, 6, 1, 6, 1.

We need to know the number of page frames available in RAM.

The more frames are available in RAM the less is the number of page faults.



FIFO Page replacement

The FIFO (First In First Out) associates with each page the time when that page was brought into memory.

e.g., the OS keeps information in the form: $[Page_{\#}, PID, t_{in}]$

When a page must be replaced, the oldest page is chosen (victim frame).

Another way to implement this, is to use a global queue that keeps track of all pages.

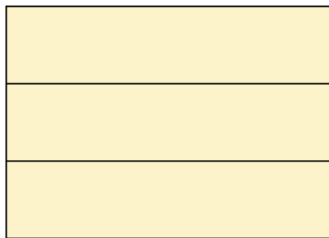
When a page needs to be swapped out, the one at the head of the queue is selected.

FIFO Page replacement

Consider the following string sequence and consider that we have 3 free frames:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

Initially all three available frames are empty:

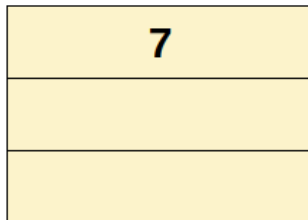


FIFO Page replacement

Consider the following string sequence and consider that we have 3 free frames:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

The first page (7) is not in the memory, a page fault occurs and page (7) is loaded:



FIFO Page replacement

Consider the following string sequence and consider that we have 3 free frames:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

The page (0) is not in the memory, a page fault occurs and page (0) is loaded:

7
0

FIFO Page replacement

Consider the following string sequence and consider that we have 3 free frames:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

The third page (1) is not in the memory, a page fault occurs and page (1) is loaded:

7
0
1

FIFO Page replacement

Consider the following string sequence and consider that we have 3 free frames:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

Now page (2) is not in the memory, a page fault occurs and page (2) is loaded replacing page (7) (oldest):

2
0
1

FIFO Page replacement

Consider the following string sequence and consider that we have 3 free frames:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

page (0) is in the memory, no page fault:

2
0
1

FIFO Page replacement

Consider the following string sequence and consider that we have 3 free frames:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

page (3) is not in the memory, a page fault occurs, page (3) is loaded replacing page (0):

2
3
1

FIFO Page replacement

Consider the following string sequence and consider that we have 3 free frames:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

page (0) is not in the memory, a page fault occurs and page (0) is loaded replacing page (1):

2
3
0

FIFO Page replacement

Consider the following string sequence and consider that we have 3 free frames:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

page (4) is not in the memory, a page fault occurs and page (4) is loaded replacing page (2):

4
3
0

FIFO Page replacement

Consider the following string sequence and consider that we have 3 free frames:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

page (2) is not in the memory, a page fault occurs and page (2) is loaded replacing page (3):

4
2
0

FIFO Page replacement

Consider the following string sequence and consider that we have 3 free frame:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

page (3) is not in the memory, a page fault occurs and page (3) is loaded replacing page (0):

4
2
3

FIFO Page replacement

Consider the following string sequence and consider that we have 3 free frames:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

page (0) is not in the memory, a page fault occurs and page (0) is loaded replacing page (4):

0
2
3

FIFO Page replacement

Consider the following string sequence and consider that we have 3 free frames:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

page (3) is in the memory, no page fault:

0
2
3

FIFO Page replacement

Consider the following string sequence and consider that we have 3 free frames:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

page (2) is in the memory, no page fault:

0
2
3

FIFO Page replacement

Consider the following string sequence and consider that we have 3 free frames:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

page (1) is not in the memory, a page fault occurs and page (1) is loaded replacing page (2):

0
1
3

FIFO Page replacement

Consider the following string sequence and consider that we have 3 free frames:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

page (2) is not in the memory, a page fault occurs and page (2) is loaded replacing page (3):

0
1
2

FIFO Page replacement

Consider the following string sequence and consider that we have 3 free frames:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

page (0) is in the memory, no page fault:

0
1
2

FIFO Page replacement

Consider the following string sequence and consider that we have 3 free frames:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

page (1) is in the memory, no page fault:

0
1
2

FIFO Page replacement

Consider the following string sequence and consider that we have 3 free frames:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

page (7) is not in the memory, a page fault occurs and page (7) is loaded replacing page (0):

7
1
2

FIFO Page replacement

Consider the following string sequence and consider that we have 3 free frames:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

page (0) is not in the memory, a page fault occurs and page (0) is loaded replacing page (1):

7
0
2

FIFO Page replacement

Consider the following string sequence and consider that we have 3 free frames:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

page (1) is not in the memory, a page fault occurs and page (1) is loaded replacing page (2):

7
0
1

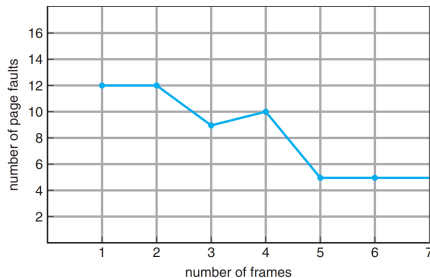
FIFO Page replacement

With that reference string, 3 free frames, and using FIFO replacement algorithm: We ended up with: **15 page faults**.

In a nutshell FIFO is easy to understand and implement, but its performance is not always good.

Example. Consider the following reference string (**Belady's anomaly**).

1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5.



Optimal Page replacement

The OPT (Optimal page replacement) a.k.a., Clairvoyant replacement algorithm, replaces the pages that will not be used for the longest period of time.

It guarantees the lowest possible page-fault rate for a fixed number of frames.

Theoretical, cannot be implemented, because it requires future knowledge of the reference string.

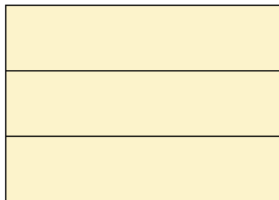
Used mainly for comparative studies (research & development).

OPT Page replacement

Consider the following string sequence and consider that we have 3 free frames:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

Initially all three available frames are empty:

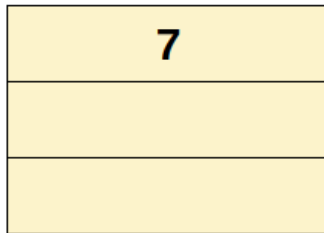


OPT Page replacement

Consider the following string sequence and consider that we have 3 free frames:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

The first page (7) is not in the memory, a page fault occurs and page (7) is loaded:



OPT Page replacement

Consider the following string sequence and consider that we have 3 free frames:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

The page (0) is not in the memory, a page fault occurs and page (0) is loaded:

7
0

OPT Page replacement

Consider the following string sequence and consider that we have 3 free frames:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

The third page (1) is not in the memory, a page fault occurs and page (1) is loaded:

7
0
1

OPT Page replacement

Consider the following string sequence and consider that we have 3 free frames:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

The fourth page (2) is not in the memory, a page fault occurs and page (2) is loaded replacing page (7):

2
0
1

OPT Page replacement

Consider the following string sequence and consider that we have 3 free frames:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

Page (0) is in the memory, no page fault:

2
0
1

OPT Page replacement

Consider the following string sequence and consider that we have 3 free frames:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

The fourth page (3) is not in the memory, a page fault occurs and page (3) is loaded replacing page (1):

2
0
3

OPT Page replacement

Consider the following string sequence and consider that we have 3 free frames:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

page (0) is in the memory, no page fault:

2
0
3

OPT Page replacement

Consider the following string sequence and consider that we have 3 free frames:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

Page (4) is not in the memory, a page fault occurs and page (4) is loaded replacing page (0):

2
4
3

OPT Page replacement

Consider the following string sequence and consider that we have 3 free frames:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

Page (2) is in the memory, no page fault:

2
4
3

OPT Page replacement

Consider the following string sequence and consider that we have 3 free frames:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

Page (3) is in the memory, no page fault:

2
4
3

OPT Page replacement

Consider the following string sequence and consider that we have 3 free frames:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

Page (0) is not in the memory, a page fault occurs and page (0) is loaded replacing page (4):

2
0
3

OPT Page replacement

Consider the following string sequence and consider that we have 3 free frames:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

Page (3) is in the memory, no page fault:

2
0
3

OPT Page replacement

Consider the following string sequence and consider that we have 3 free frames:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

Page (2) is in the memory, no page fault:

2
0
3

OPT Page replacement

Consider the following string sequence and consider that we have 3 free frames:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

Page (1) is not in the memory, page fault, page (1) loaded replacing page (3):

2
0
1

OPT Page replacement

Consider the following string sequence and consider that we have 3 free frames:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

Page (2) is in the memory, no page fault:

2
0
1

OPT Page replacement

Consider the following string sequence and consider that we have 3 free frames:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

Page (0) is in the memory, no page fault:

2
0
1

OPT Page replacement

Consider the following string sequence and consider that we have 3 free frames:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

Page (1) is in the memory, no page fault:

2
0
1

OPT Page replacement

Consider the following string sequence and consider that we have 3 free frames:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

Page (7) is not in the memory, page fault, page (7) loaded replacing page (2):

7
0
1

OPT Page replacement

Consider the following string sequence and consider that we have 3 free frames:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

Page (0) is in the memory, no page fault:

7
0
1

OPT Page replacement

Consider the following string sequence and consider that we have 3 free frames:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

Page (1) is in the memory, no page fault:

7
0
1

OPT Page replacement

Consider the following string sequence and consider that we have 3 free frames:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

Page (1) is in the memory, no page fault:

7
0
1

This time: 9 page faults. (**Twice as good as FIFO**)

LRU Page replacement

The LRU (Least Recently Used page replacement) replaces the pages that had not been referenced for the longest period of time.

e.g., the OS keeps information in of form: $[Page_i, t_{last}]$ in the page table.

It allows a better page-fault rate than FIFO, and does not suffer from **Belady's anomaly**.

Can be perceived as OPT looking backward in time, rather than forward.

Requires substantial hardware assistance.

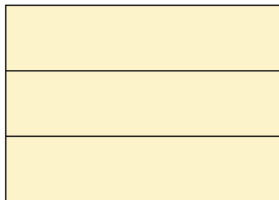
- Can use a logical clock that is incremented after each reference.
- Use a stack of pages (the most recently used page is always on the top).

LRU Page replacement

Consider the following string sequence and consider that we have 3 free frames:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

Initially all three available frames are empty:

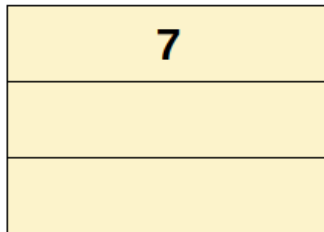


LRU Page replacement

Consider the following string sequence and consider that we have 3 free frames:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

The first page (7) is not in the memory, a page fault occurs and page (7) is loaded:



LRU Page replacement

Consider the following string sequence and consider that we have 3 free frames:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

The page (0) is not in the memory, a page fault occurs and page (0) is loaded:

7
0

LRU Page replacement

Consider the following string sequence and consider that we have 3 free frames:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

The third page (1) is not in the memory, a page fault occurs and page (1) is loaded:

7
0
1

LRU Page replacement

Consider the following string sequence and consider that we have 3 free frames:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

Page (2) is not in the memory, a page fault occurs and page (7) is replaced to host page (2):

2
0
1

LRU Page replacement

Consider the following string sequence and consider that we have 3 free frames:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

Page (0) is in the memory, no page fault:

2
0
1

LRU Page replacement

Consider the following string sequence and consider that we have 3 free frames:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

Page (3) is not in the memory, a page fault occurs and page (1) is replaced to host page (3):

2
0
3

LRU Page replacement

Consider the following string sequence and consider that we have 3 free frames:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

Page (0) is in the memory, no page fault:

2
0
3

LRU Page replacement

Consider the following string sequence and consider that we have 3 free frames:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

Page (4) is not in the memory, a page fault occurs and page (2) is replaced to host page (4):

4
0
3

LRU Page replacement

Consider the following string sequence and consider that we have 3 free frames:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

Page (2) is not in the memory, a page fault occurs and page (3) is replaced to host page (2):

4
0
2

LRU Page replacement

Consider the following string sequence and consider that we have 3 free frames:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

Page (3) is not in the memory, a page fault occurs and page (0) is replaced to host page (3):

4
3
2

LRU Page replacement

Consider the following string sequence and consider that we have 3 free frames:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

Page (0) is not in the memory, a page fault occurs and page (4) is replaced to host page (0):

0
3
2

LRU Page replacement

Consider the following string sequence and consider that we have 3 free frames:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

Page (3) is in the memory, no page fault:

0
3
2

LRU Page replacement

Consider the following string sequence and consider that we have 3 free frames:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

Page (2) is in the memory, no page fault:

0
3
2

LRU Page replacement

Consider the following string sequence and consider that we have 3 free frames:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

Page (1) is not in the memory, a page fault occurs and page (0) is replaced to host page (1):

1
3
2

LRU Page replacement

Consider the following string sequence and consider that we have 3 free frames:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

Page (2) is in the memory, no page fault:

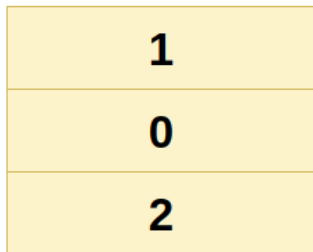
1
3
2

LRU Page replacement

Consider the following string sequence and consider that we have 3 free frames:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

Page (0) is not in the memory, a page fault occurs and page (3) is replaced to host page (0):



LRU Page replacement

Consider the following string sequence and consider that we have 3 free frames:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

Page (1) is in the memory, no page fault:

1
0
2

LRU Page replacement

Consider the following string sequence and consider that we have 3 free frames:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

Page (7) is not in the memory, a page fault occurs and page (2) is replaced to host page (7):

1
0
7

LRU Page replacement

Consider the following string sequence and consider that we have 3 free frames:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

Page (0) is in the memory, no page fault:

1
0
7

LRU Page replacement

Consider the following string sequence and consider that we have 3 free frames:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

Page (1) is in the memory, no page fault:

1
0
7

LRU Page replacement

Consider the following string sequence and consider that we have 3 free frames:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

1
0
7

This time: 12 page faults. **(Better than FIFO)**

Other Page replacement algorithms

You can find other algorithms in the literature such as:

- **LIFO (Last In First Out)**. Replace the last page that was swapped in (inverse of FIFO).
- **RANDOM**. Randomly select a page without any preference.
- **LFU (Least Frequently Used)**. Keep in memory pages that are mostly used (large reference count).
- **MFU (Most Frequently Used)**. Keep in memory pages that have a small reference count (assumption: pages just brought into RAM).

Global Vs Local Allocation

When a page fault occurs while there is no free frames available in the memory, then depending on the page-replacement algorithm:

The victim page may be local to the process causing the fault or global

Global replacement Algo. In this class of algorithms, the victim frame is selected from the set of all existing frames in the memory.

The number of frames for a high-priority process can increase: Consider taking frames from lower-priority processes

Local replacement Algo. Here the victim frame is selected from the set of frames that belong to the process which caused the page fault.

The number of frames for a process does not change (fixed #).

- End.